ZYNQ 开发平台 HLS 教程

版本号：V1.03

2019-03-03 13:09:10

版权声明

Copyright ©2012-2018 芯驿电子科技（上海）有限公司

公司网址:
Http://www.alinx.com.cn

技术论坛:
http://www.heijin.org

官方旗舰店:
http://alinx.jd.com

邮箱:
avic@alinx.com.cn

电话:
021-67676997

传真:
021-37737073

ALINX 微信公众号:



前言

Vivado HLS 能提高系统设计的抽象层次，为设计人员带来切实的帮助。Vivado HLS 通过下面两种方法提高抽象层次：第一，使用 C/C++作为编程语言，充分利用该语言中提供的高级结构。第二，提供更多数据原语，便于设计人员使用基础硬件构建块（位向量、队列等）。与使用 RTL 相比，这两大特性有助于设计人员使用 Vivado HLS 更轻松地解决常见的协议系统设计难题。最终简化系统汇编，简化 FIFO 和存储器访问，实现控制流程的抽象。

Vivado HLS 便于架构研究，用户只需在代码中插入程序指令（如使用 GUI 或批处理模式时的 Tcl 命令），就可以把设计所需特性传递给综合工具。这样用户可以在不修改设计代码本身的情况下研究大量备选架构方案。研究的范围可以是模块流水线化等根本性问题，也可以是 FIFO 队列深度等较常见的问题。

Vivado HLS 便于仿真。C 和 RTL 仿真是 Vivado HLS 另一个大放异彩的地方。设计一般采用两步流程验证：第一步是 C 语言仿真。这个步骤中 C/C++的编译和执行与常见的 C/C++程序相同；第二步是 C/RTL 协仿真。在这步骤中，Vivado HLS 会根据 C/C++测试平台自动生成 RTL 测试平台，然后设置并执行 RTL 仿真，检查实现方案吧的正确性。

如能充分发挥这些优势，这将对于用户的系统设计大有裨益。这不仅体现在开发时间和生产力上，还由于 Vivado HLS 代码更加紧凑的特点，体现在代码可维护性和可读性上。此外通过高层次综合，用户仍能有效控制架构及其特性。正确理解和使用 Vivado HLS 程序对实现这一控制起着根本作用。

目录

版权声明 2
前言 3
目录 4
准备工作及注意事项 7
软件环境 7
硬件环境 7
实验工程及目录说明 10
实验快速复现 11
工程重新编译注意事项 11
第一章 初识 HLS 13
1.1 实验 led 控制 13
1.1.1 创建 vivado hls 工程 13
1.1.2 创建 vivado 工程 17
1.1.3 实验总结 28
1.2 工程路径 28
1.3 HLS 简介 29
1.3.1 Vivado HLS 包含库 29
1.3.2 Vivado HLS 接口 30
1.3.3 hls 官方教程 30
第二章 状态指示 led 31
2.1 模块控制 block-level 31
2.2 可配置的模块 32
2.3 工程路径 32
2.4 实验结果 33
第三章 浮点协处理 34
3.1 实验介绍 34
3.2 IP 创建 34
3.2.1 HLS 源代码 34
3.2.2 接口介绍 34
3.2.3 运算 35
3.2.4 其它说明 35
3.3 TestBench 35
3.3.1 应用程序创建 35
3.3.2 源代码 35
3.3.3 C 仿真 36

3.3.4 RTL 仿真 36
3.4 工程路径 36
3.5 运行结果 36
第四章 视频彩条 38
4.1 Vivado HLS 视频开发 38
4.1.1 与 OpenCV 关系 38
4.1.2 VivadoHLS 视频库函数 38
4.2 实验介绍 39
4.3 HLS IP 创建 39
4.3.1 源代码 39
4.3.2 接口介绍 40
4.3.3 hls::Mat 介绍 41
4.3.4 优化 41
4.4 工程路径 41
4.5 实验结果 41
第五章 视频帧缓存读写管理 43
5.1 实验介绍 43
5.2 模块主要代码 43
5.1 工程路径 44
5.2 实验结果 44
第六章 图像缩放叠加 46
6.1 实验介绍 46
6.2 模块主要代码 46
6.3 工程路径 47
6.4 实验结果 48
第七章 字符叠加 49
7.1 实验介绍 49
7.2 模块主要代码 49
7.3 工程路径 50
7.4 实验结果 50
第八章 图像对比度调整 51
8.1 实验介绍 51
8.2 模块主要代码 51
8.3 工程路径 52
8.4 实验结果 52
第九章 自动聚焦 54
9.1 实验介绍 54
9.2 代码 54

9.3 工程路径 55
9.4 实验结果 55
第十章 边缘检测 57
10.1 实验介绍 57
10.2 TestBench 结果 57
10.3 模块主要代码 59
10.4 工程路径 60
10.5 实验结果 60
第十一章 角点检测 62
11.1 模块主要代码 63
11.2 工程路径 64
11.3 实验结果 64
第十二章 快速傅里叶变换 FFT 65
12.1 实验介绍 65
12.2 模块主要代码 65
12.3 工程路径 66
12.4 实验结果 66

准备工作及注意事项

本教程基于学习者已经熟练使用 Vivado 和 xSDK 的情况下编写，不详细讲解 Vivado 工程建立，xSDK 裸机软件编写，中断处理等。HLS 要求学习者熟练使用 C++，具备一定硬件思维，对硬件流水线、组合逻辑、时序逻辑、FIFO、RAM、AXI 总线协议有深刻理解才能学习，免得出现低级错误。

软件环境

本教程基于软件版本 SDx 2017.4，这里要用到的快捷方式图标有：



vivado 软件

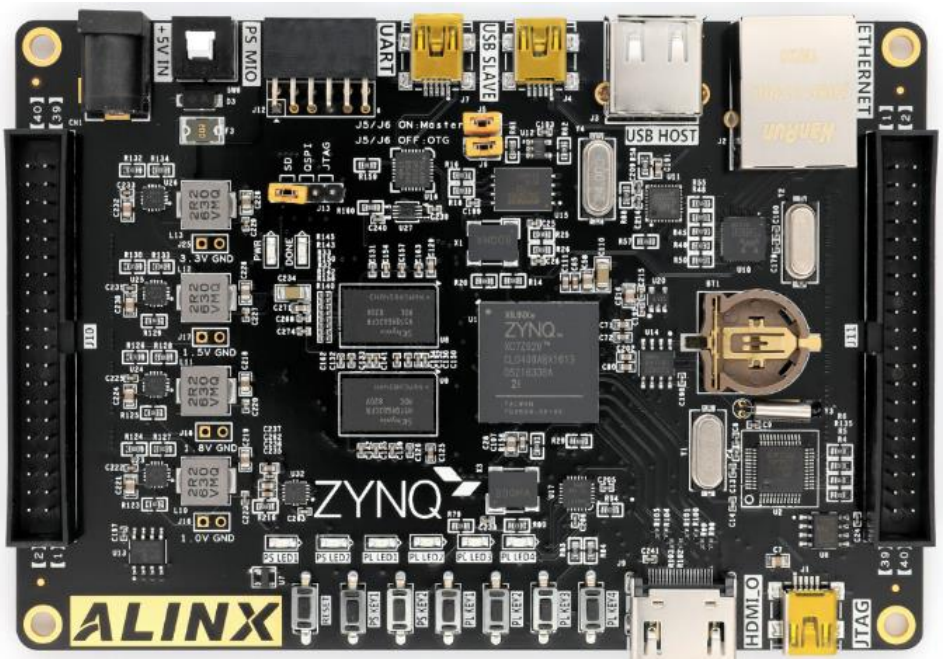


vivado hls 软件

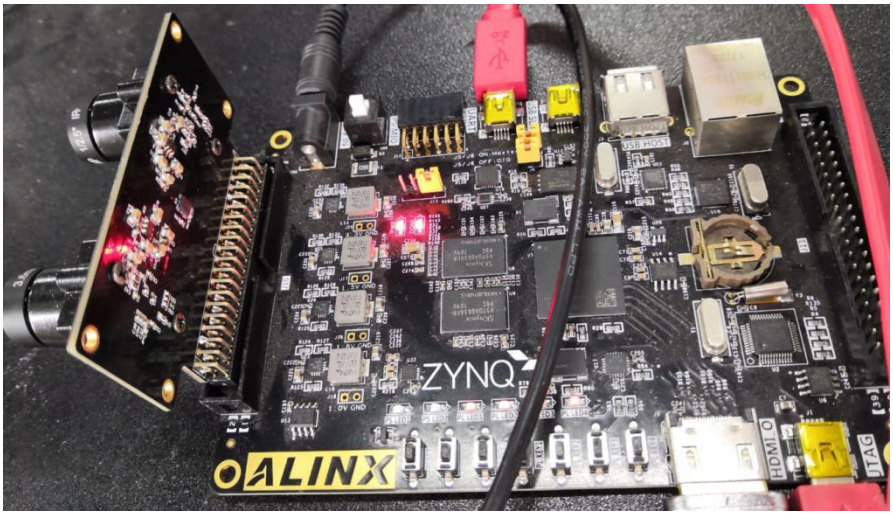
硬件环境

ZYNQ（ALINX 芯驿电子科技）开发板一块，双目镜头模块一个。当前支持的型号列表如下：

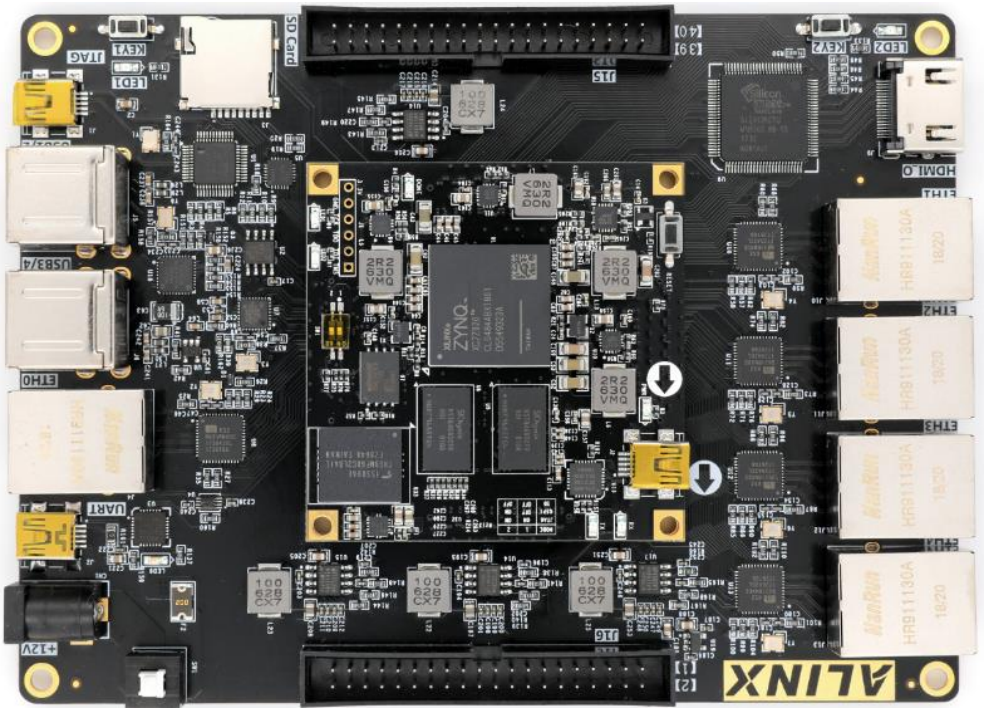
开发板型号	双目模块	ZYNQ 芯片型号	ddr 参数兼容型号
AX7020	AN5642	xc7z020clg400-2	MT41J256M16 RE-125
AX7021	AN5642	xc7z020clg484-2	MT41J256M16 RE-125
AX7015	AN5642	xc7z015clg485-2	MT41J256M16 RE-125
AX7350	FL0214	xc7z035ffg676-2	MT41J256M16 RE-125
AX7Z035	AN5642	xc7z035ffg676-2	MT41J256M16 RE-125
AX7Z100	AN5642	xc7z100ffg900-2	MT41J256M16 RE-125



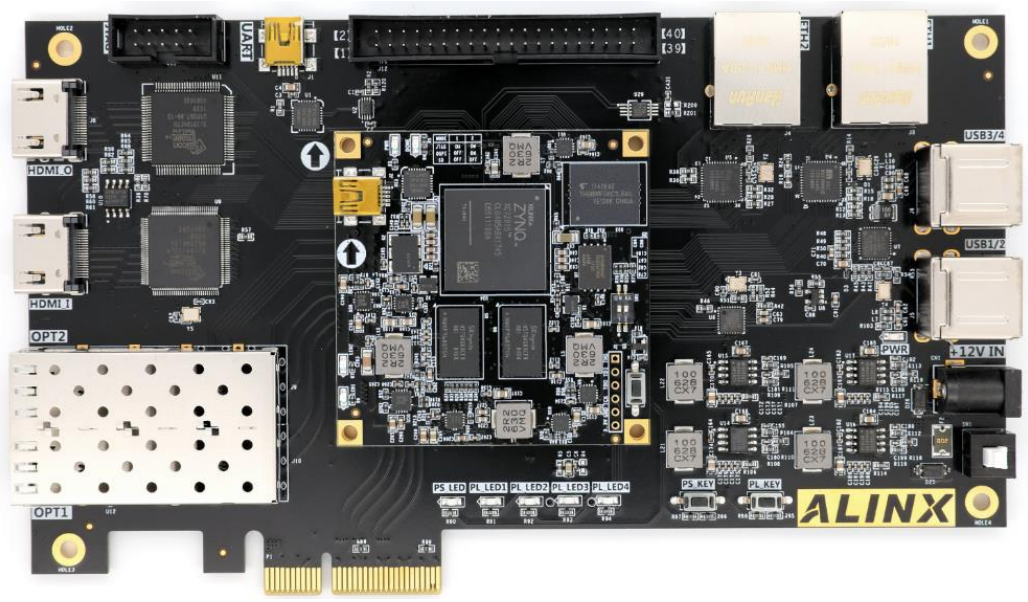
AX7020



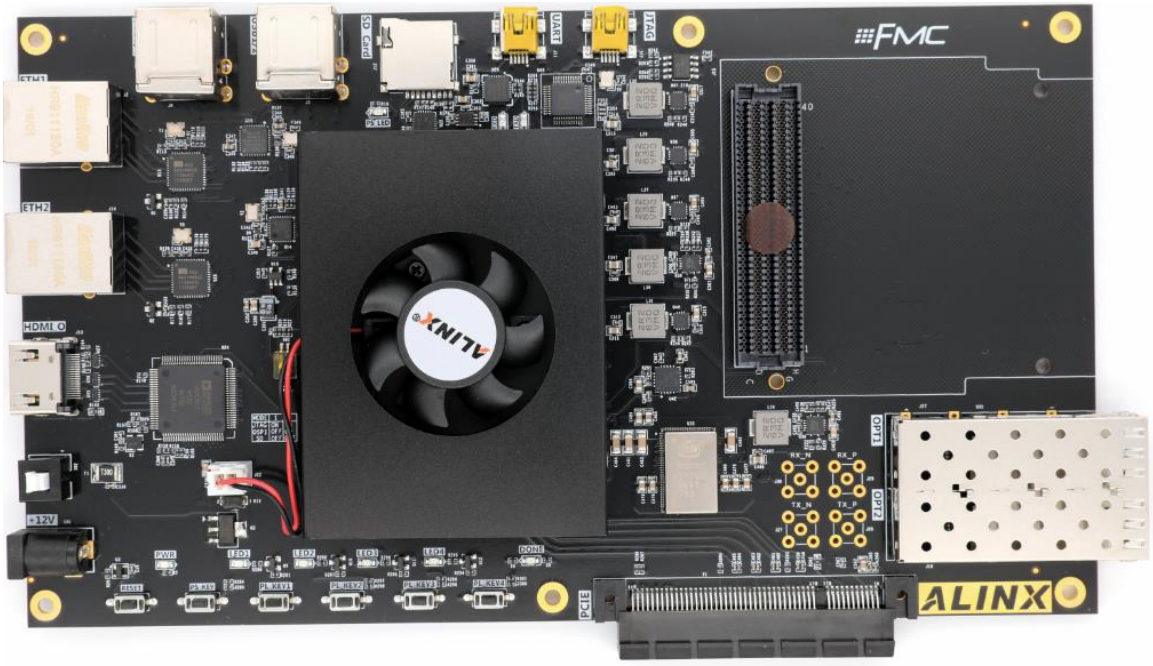
AX7020 正确连接摄像头方式，例程中都是插在 J10



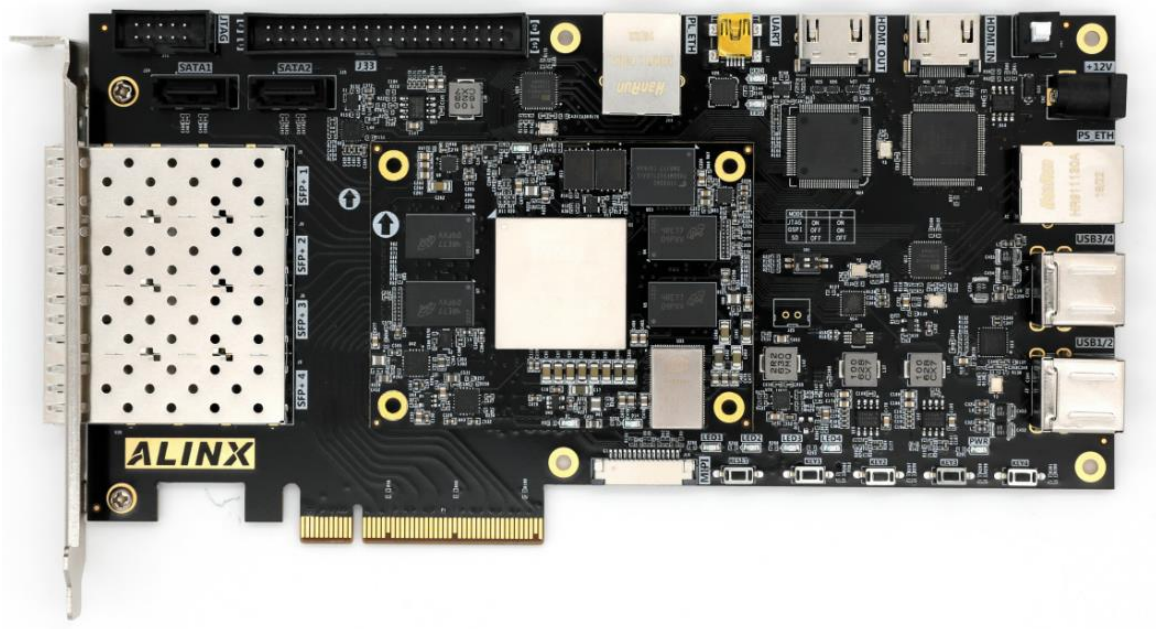
AX7021



AX7015



AX7350



AX7Z035/AX7Z100

另外需要自己准备好 HDMI 显示器一台（支持 1080P60）。

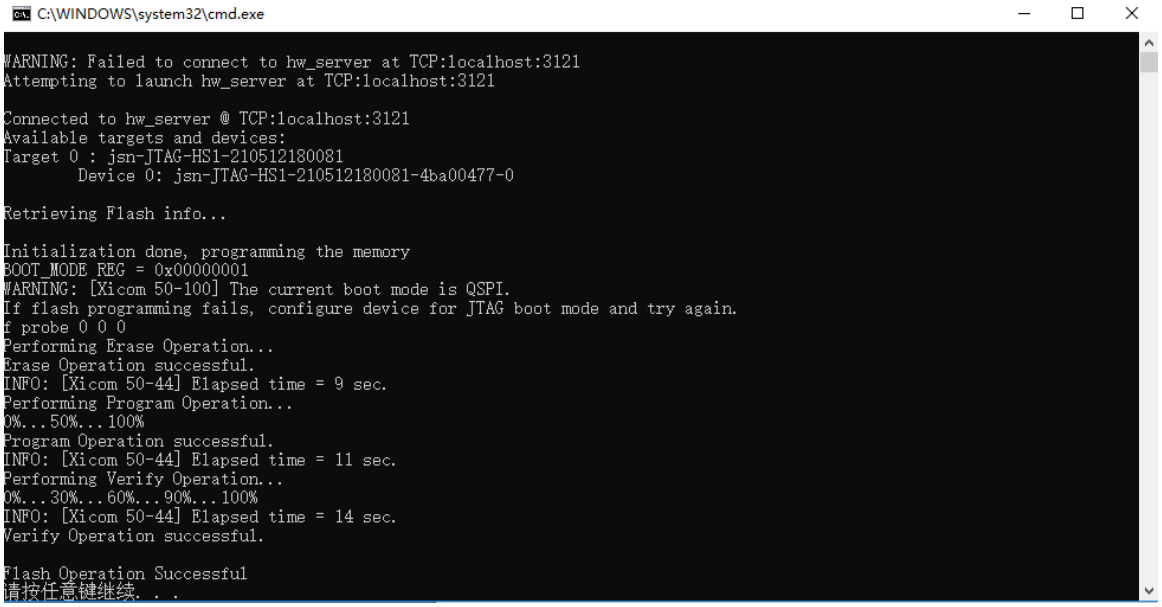
实验工程及目录说明

工程名称	目录路径	说明
HLS 工程	hls	各算法导出 IP，由 vivado 引用
vivado 工程	vivado	
SDK 工程	vivado/xx_project/xx_.sdk/	软件程序
常用 ip	vivado/xx_project/ip/normal	Verilog 封装 ip

HLS 导出 IP	vivado/xx_project/ip/hls	HLS 导出 ip 后拷贝至该目录
flash_load_comm	flash_load_comm	qspi 烧录

实验快速复现

1) 连接好 jtag 接口
2) 设置启动模式为 QSPI 启动
3) 将对应实验 SDK 工程中文件 BOOT.bin 拷贝到 flash_load_comm 目录下。
4) 开发板上电
5) 点击 program_qspi.bat 脚本运行。烧录成功后界面如下

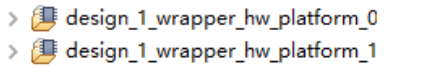


6) 开发板重新上电

   注：若 SDx 软件安装位置非默认 C 盘，则 program_qspi.bat 运行会出错，此时需要修改 program_qspi.bat 中的命令路径。

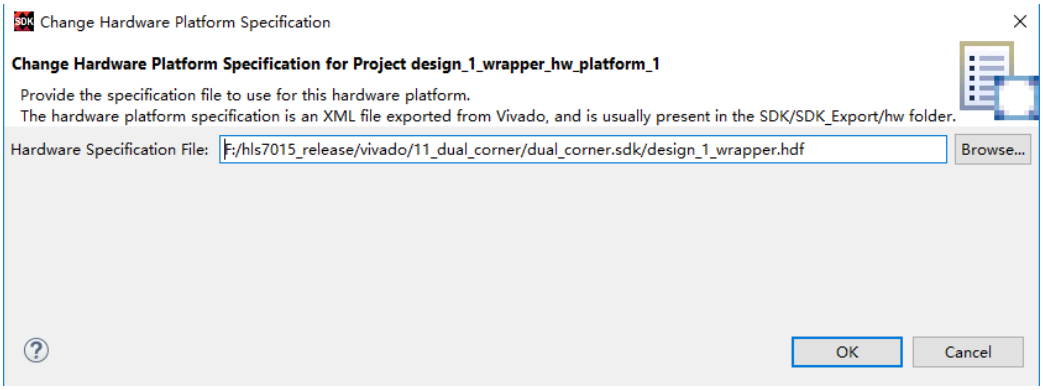
工程重新编译注意事项

1) HLS 工程修改并导出 ip 后，ip 所在路径为：hls/xx_project/solution1/impl/ip/xx.zip，使用时需要将其解压至相应目录下。
2) 通过 vivado 打开 SDK 工程，若工程路径变化后（拷贝，移动），容易多出一个硬件平台，如下图



此时，vivado 修改后导出可能使 SDK 编译混乱，所以这里，我们需要删除多出来的硬件平台目录。

a) 右键“design_1_wrapper_hw_platform_1”,在菜单中选择“Change Hardware Platform Specification”,点击“Yes”跳过警告，至如下界面



b) 拷贝当前编译框中的路径.

c) 相同方法打开“design_1_wrapper_hw_platform_0”设置界面，并粘贴之前的路径，并点击“OK”。

d) 右键“design_1_wrapper_hw_platform_1”,点击“Delete”按钮，

e) 点击“OK”按钮确认

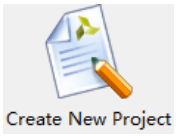
f) 点击“continue”按钮

第一章 初识 HLS

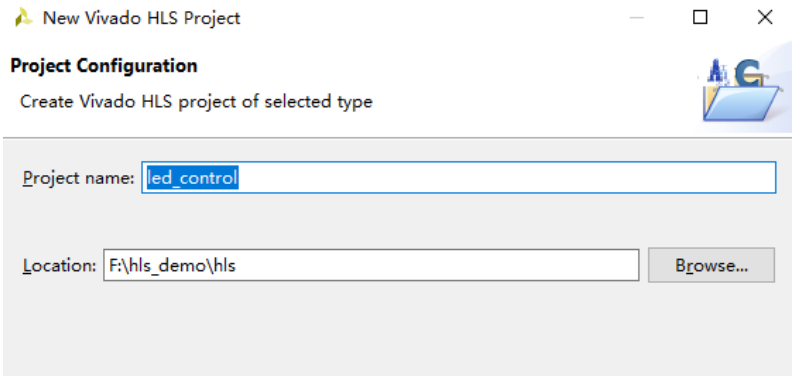
1.1 实验 led 控制

1.1.1 创建 vivado hls 工程

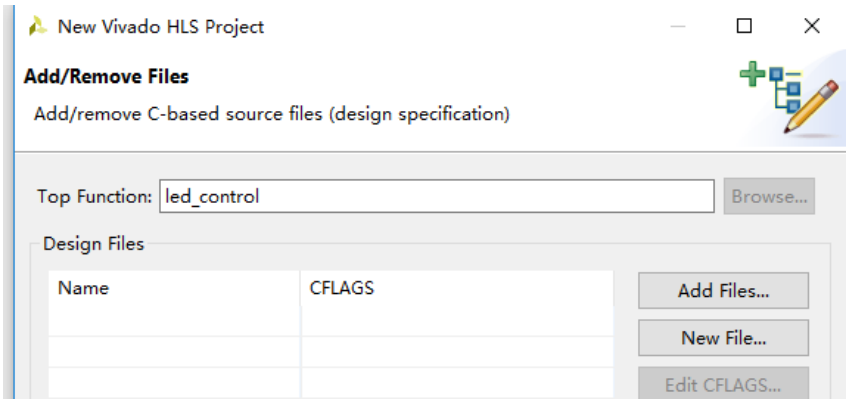
1) 打开 vivado hls 软件

2) 点击图标 

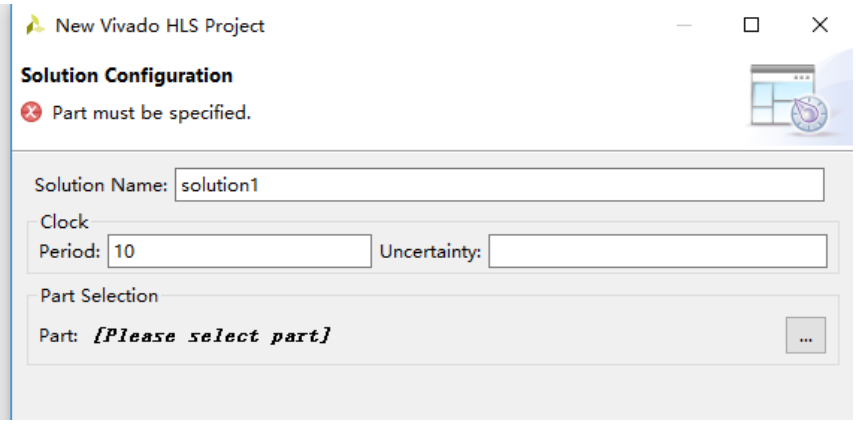
3) 设置工程名称及选择存放路径，如图



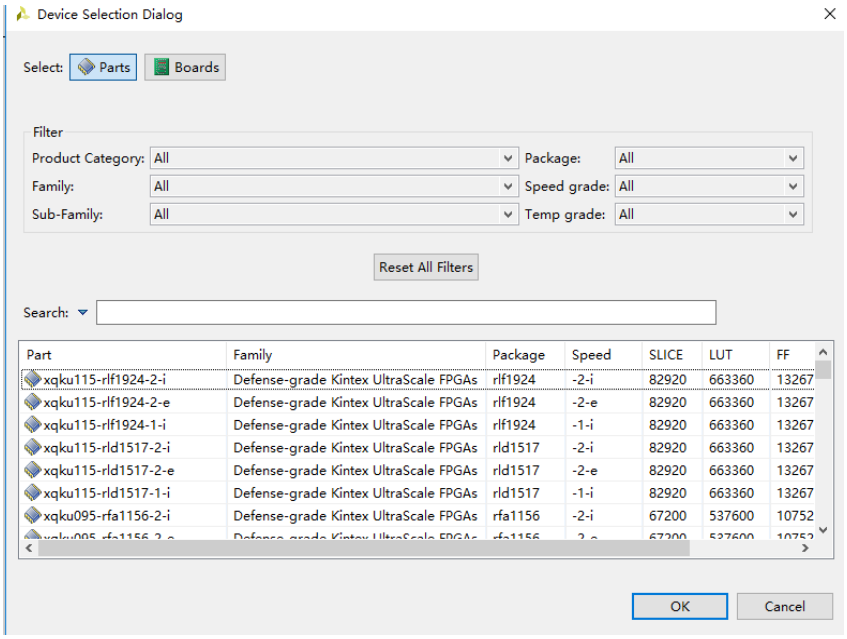
4) 设置顶层函数名，如图



5) Next→Next 至如下界面，在 Clock Period 一栏可填入运行时钟的周期，软件会根据此时钟进行逻辑优化，可根据实际需要填写，默认是 10ns，clock uncertainty 默认为时钟周期的 12.5%。



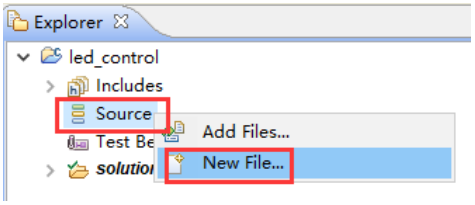
6) 点击图标...，至如下界面



7) Search 后输入框中输入 ZYNQ 芯片型号，在下拉列表框中选中对应器件，点击“OK”按钮

8) 点击“Finish”按钮，创建 vivado hls 工程完成

9) 选择”Source“图标右键，选择“New File...”，文件名设置为”lec_control.cpp“，点击”保存“



10) 在“lec_control.cpp“文件中输入如下内容

```
#include <ap_int.h>

void led_control(ap_int<1> &led)
{
#pragma HLS interface ap_none port=led
#pragma HLS interface ap_ctrl_none port=return
     for(int i=0; i<50000000; i++)
     {
          led = (i<50000000/2)?0:1;
     }
}
```

11) “Ctrl+S”保存当前文档内容

12) 点击图标 进行综合，或者点击菜单栏 Solution→Run C Synthesis→Active Solution

13) “Console”栏中输出“Finished C synthesis. “，综合完成，软件会自动跳出综合报告。在 Timing 信息中，可以看到时钟设置为 10ns，uncertainty 为 1.25ns，软件将目标时钟周期设置为 10-1.25=8.75ns。Estimated 时钟周期（最差路径）为 2.06ns，满足 8.75ns 时钟需求。Latency 需要多少周期输出结果，latency 比 loop 循环多一个周期，为迭代延迟。

Performance Estimates

Timing (ns)

Summary

Clock	Target	Estimated	Uncertainty
ap_clk	10.00	2.06	1.25

Latency (clock cycles)

Summary

Latency		Interval		
min	max	min	max	Type
50000001	50000001	50000001	50000001	none

Detail

Instance

Loop

	Latency			Initiation Interval			
Loop Name	min	max	Iteration Latency	achieved	target	Trip Count	Pipelined
- Loop 1	50000000	50000000	1	-	-	50000000	no

资源利用评估表示预估会占用多少资源，RTL 的综合可能会实现更多优化，与下表资源利用可能并不一样。

Utilization Estimates

Summary

Name	BRAM_18K	DSP48E	FF	LUT
DSP	-	-	-	-
Expression	-	-	0	69
FIFO	-	-	-	-
Instance	-	-	-	-
Memory	-	-	-	-
Multiplexer	-	-	-	24
Register	-	-	28	-
Total	0	0	28	93
Available	1000	900	343800	171900
Utilization (%)	0	0	~0	~0

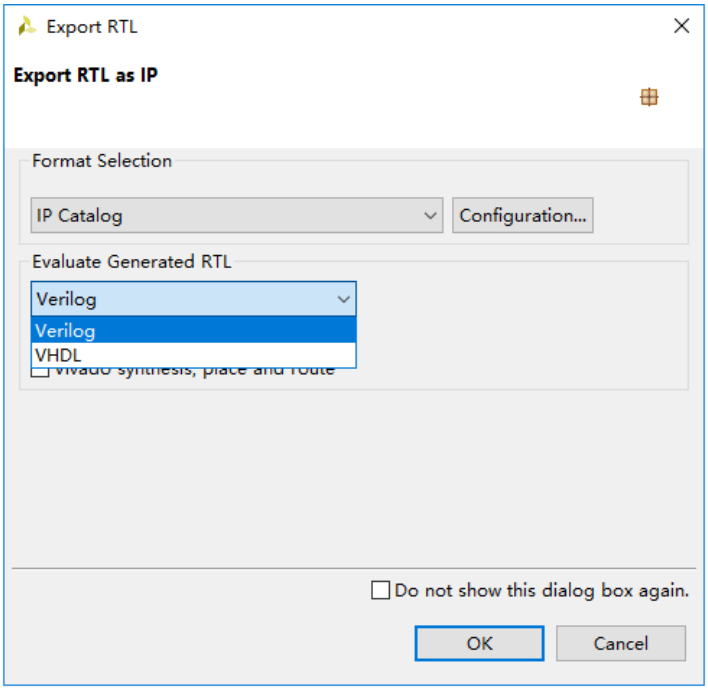
生成相应的 RTL 接口如下所示：

Interface

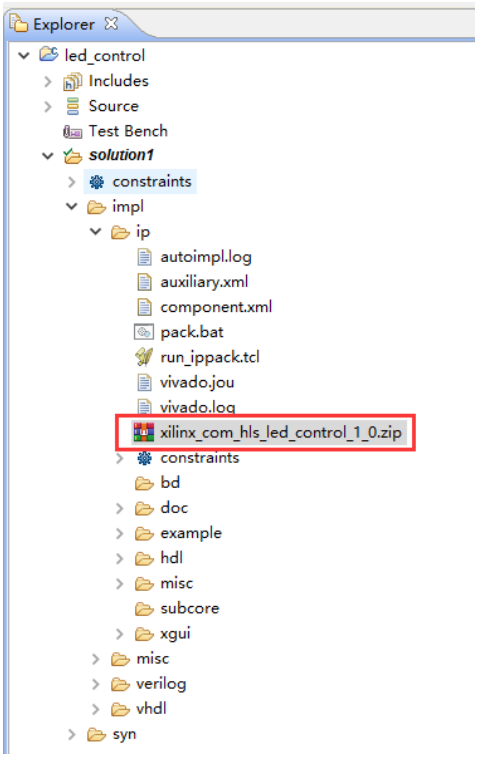
Summary

RTL Ports	Dir	Bits	Protocol	Source Object	C Type
ap_clk	in	1	ap_ctrl_none	led_control	return value
ap_rst	in	1	ap_ctrl_none	led_control	return value
led_V	out	1	ap_none	led_V	pointer

14) 点击图标，弹出如下对话框，可以在下拉框内选择需要导出的 RTL 类型。

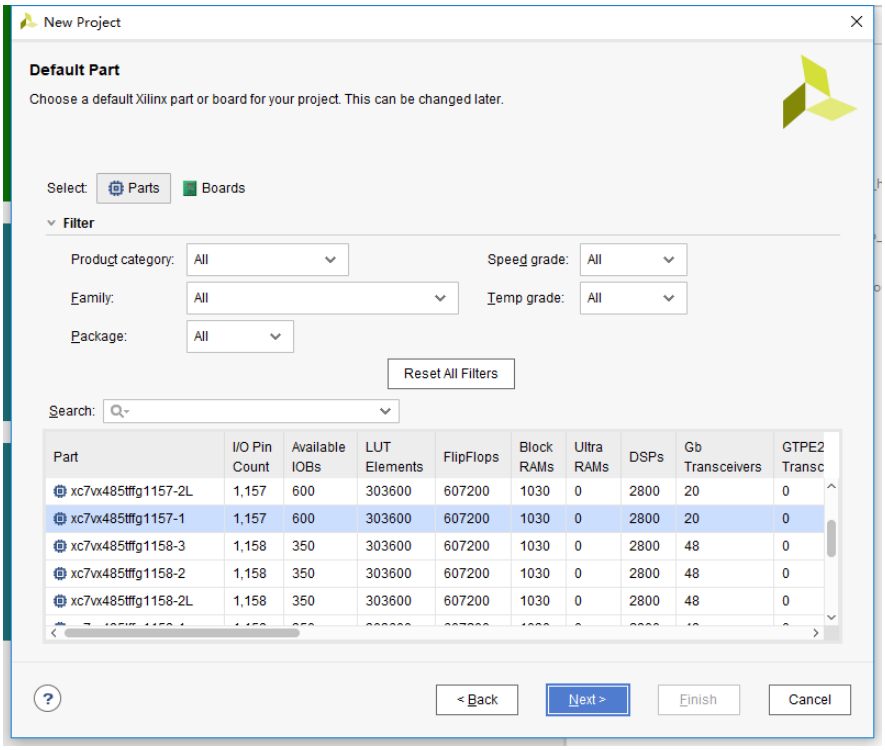


15) “Console”栏中输出“Finished export RTL. “，导出 IP 完成。在 solution1/impl/ip 文件夹能够找到生成的 IP 压缩包，可在 Vivado 中导入。



1.1.2 创建 vivado 工程

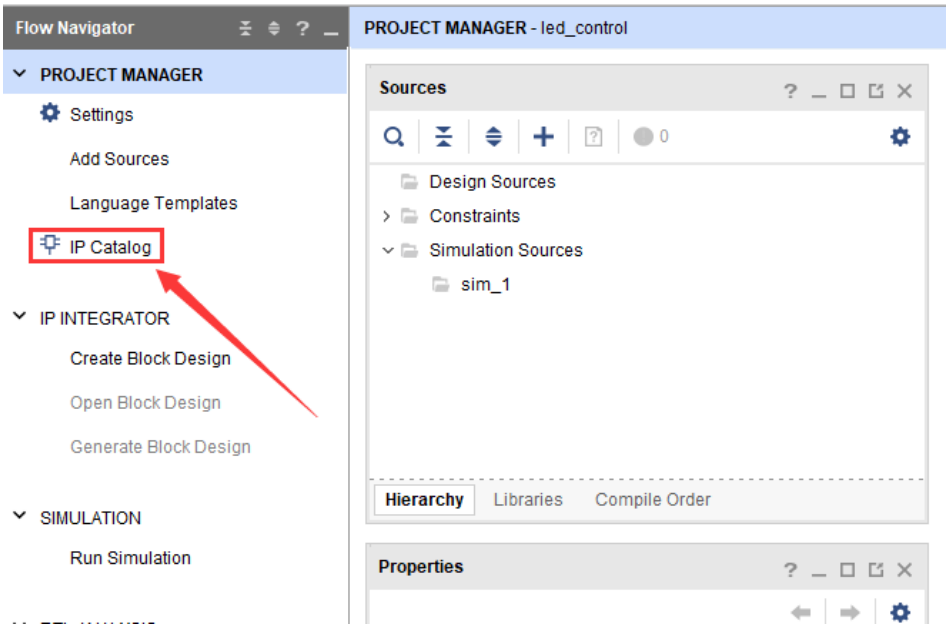
1) 打开 vivado 软件
2) 点击“Create Project“标签，弹出工程创建向导
3) Next
4) 设置存放路径和工程名
5) Next→Next→ Next→ Next，至如下界面



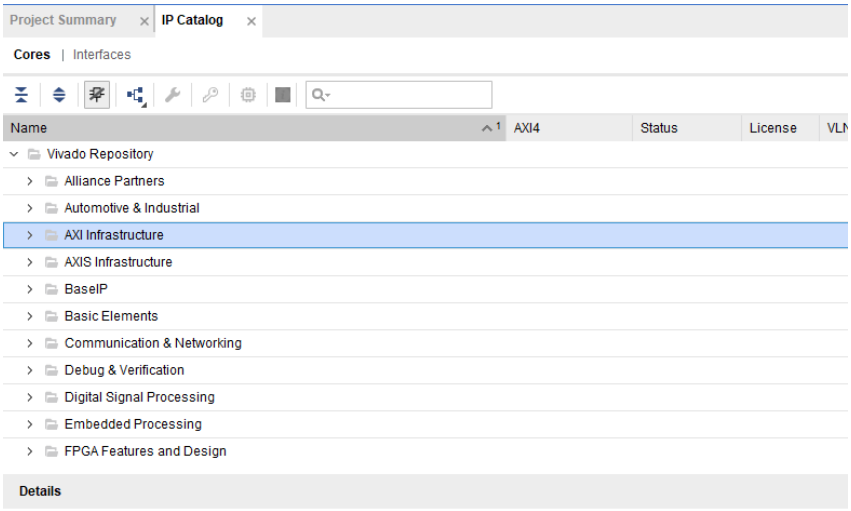
6) Search 后输入框中输入 ZYNQ 芯片型号，在下拉列表框中选中对应器件，
7) Next→Finish，完成 vivado 工程创建
8) 将 hls 工程的 IP 压缩包解压到 vivado 工程目录下。本教程解压到新建的 ip 文件夹中。



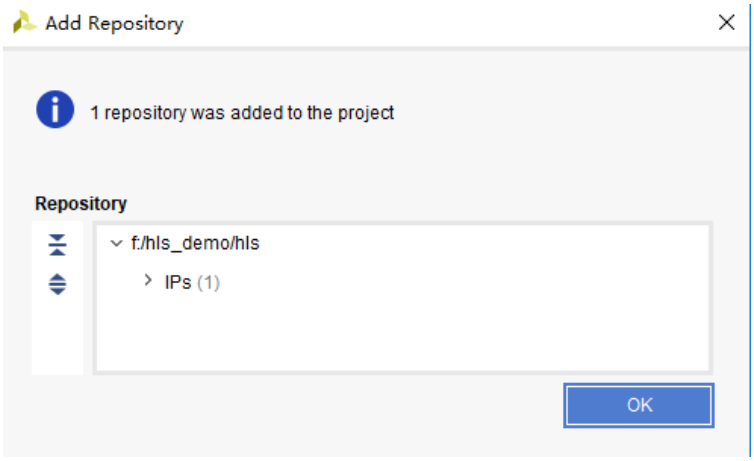
9) 进入到 vivado 软件工程界面，点击如图 “IP Catalog”标签



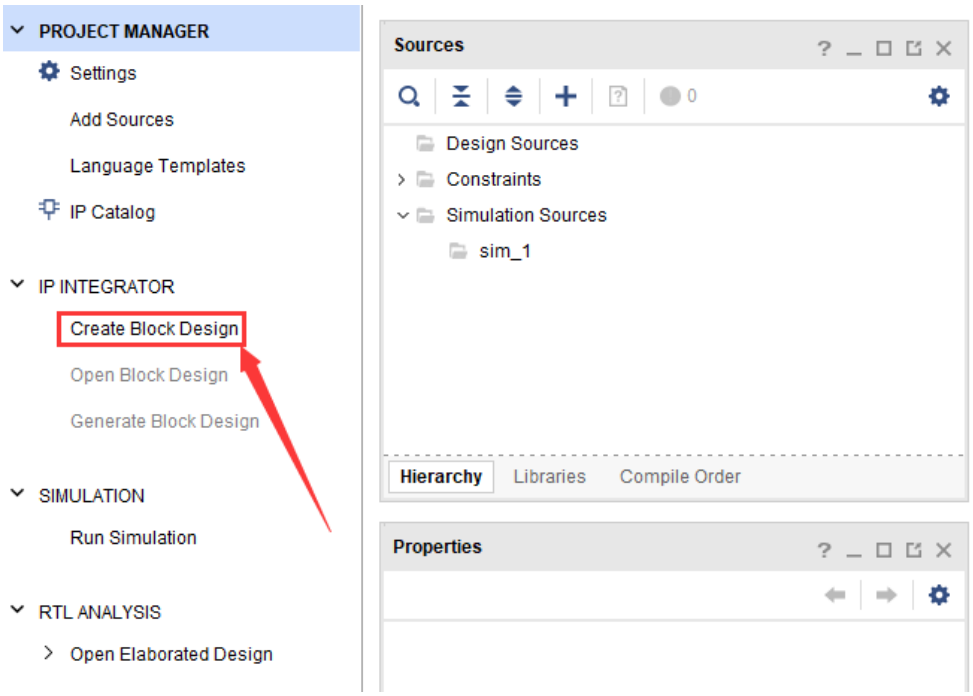
10) 在下图界面中，右键“Vivado Repository”标签，选择 “Add Repositories…”，选择 IP 所在路径，点击按钮”Select”



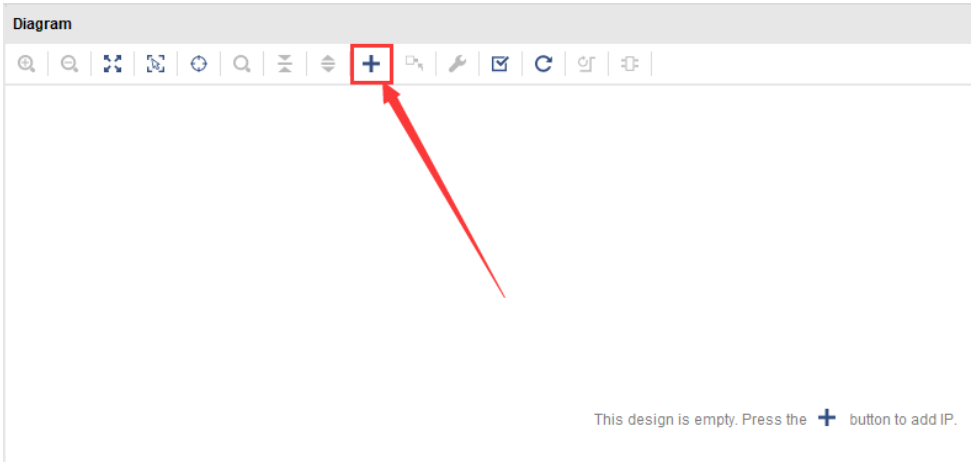
11) 弹出如下界面，点击”OK”，完成 led_control IP 添加



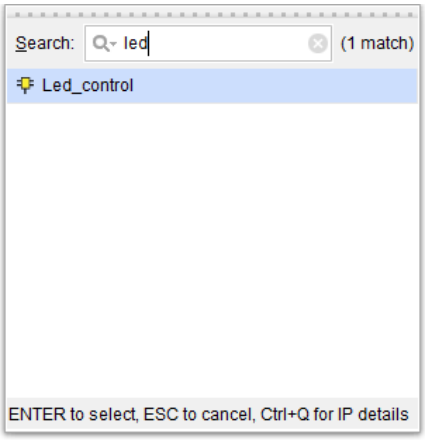
12) 点击图标”Create Block Design”，弹出“Create Block Design”对话框，选择” OK “按钮



13) 在如下界面中，点击如图所示图标，



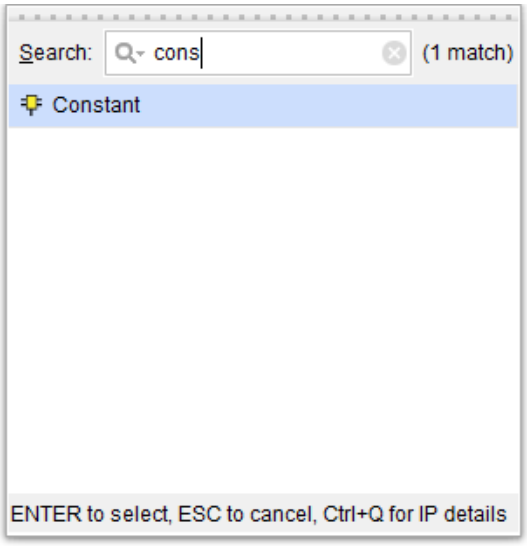
14) 在弹出的对话框中，输入“led_control”，双击检索出的 IP



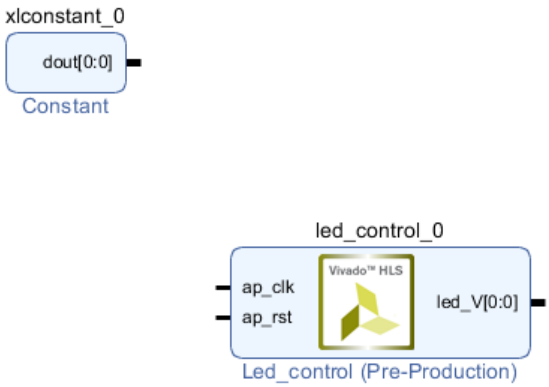
15) 成功在我们设置中，添加了我们设计的 IP，如下图



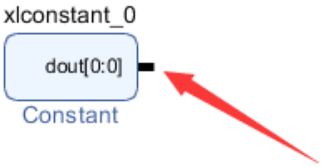
16) 点击图标 ，输入“cons”，双击检索出的 IP



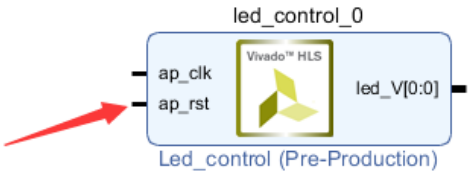
17) 此时我们的设计，如下图



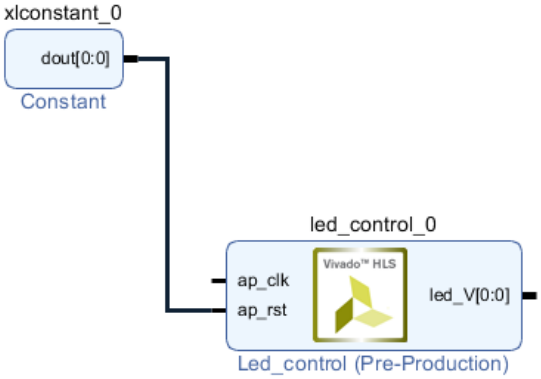
18) 鼠标移至图示管脚上，此时鼠标变成笔状，



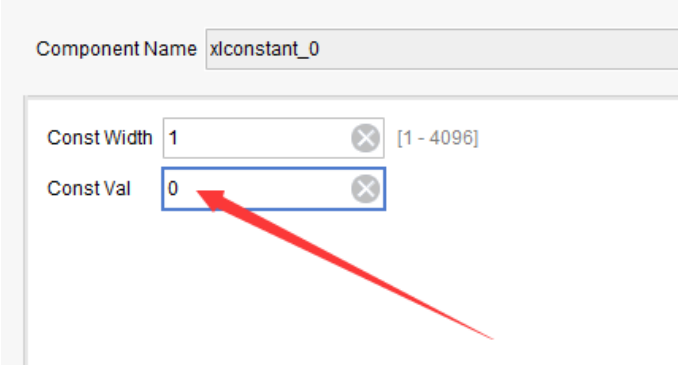
19) 左键鼠标不放，拖动鼠标至下图管脚，会发现两个管脚之间出现连线，此时松开鼠标



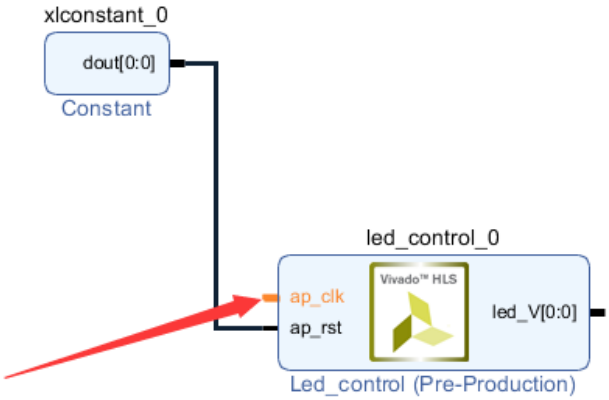
20) 此时我们的设计，如下图



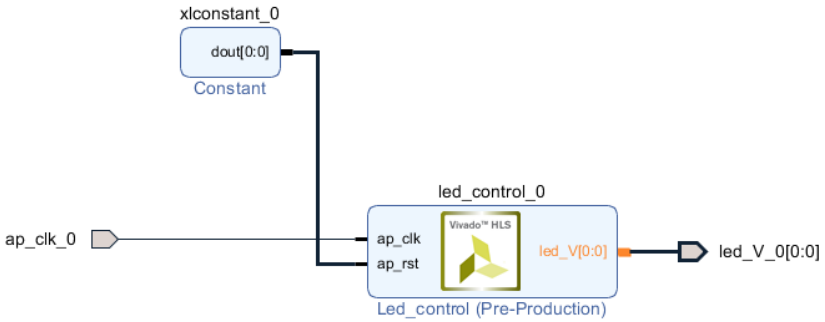
21) 双击图中” xlconstant_0“ IP，在弹出的对话框中，修改“Const val”值为 0，点击 “OK”。



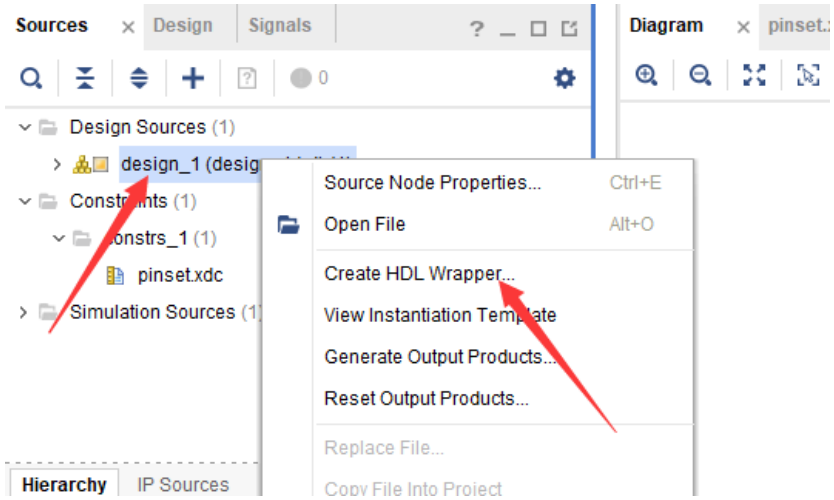
22) 鼠标点击“ap_clk”，选中该管脚，如图所示



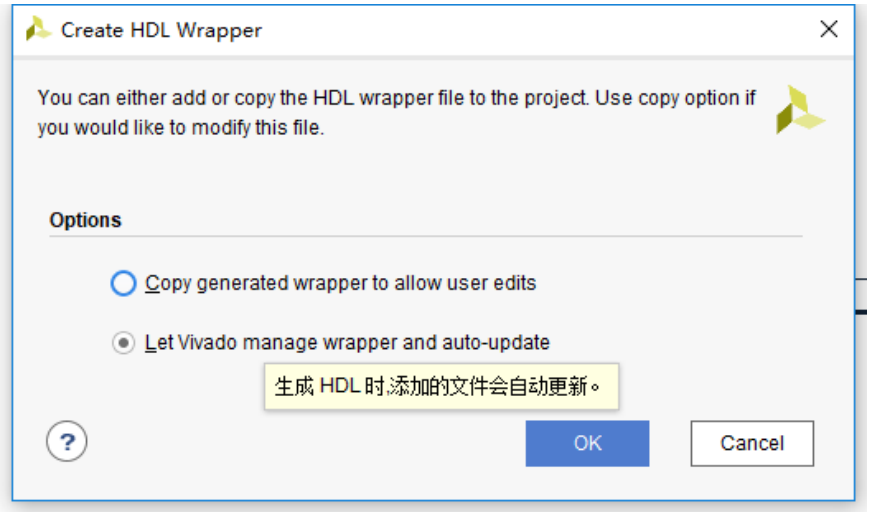
23) 此时，按”Ctl+T”，相同操作 “led_V[0:0]“管脚，此时设计如下图



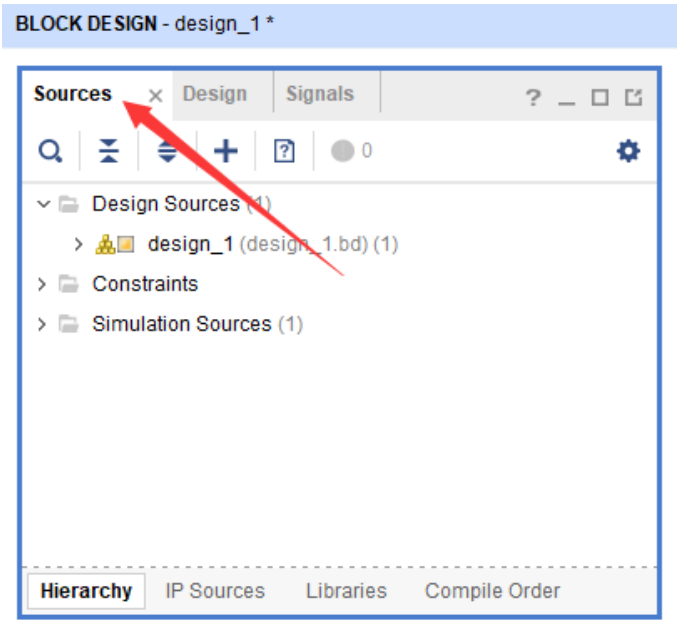
24) 右键 “design_1”，选择 “Create HDL Wrapper…”。



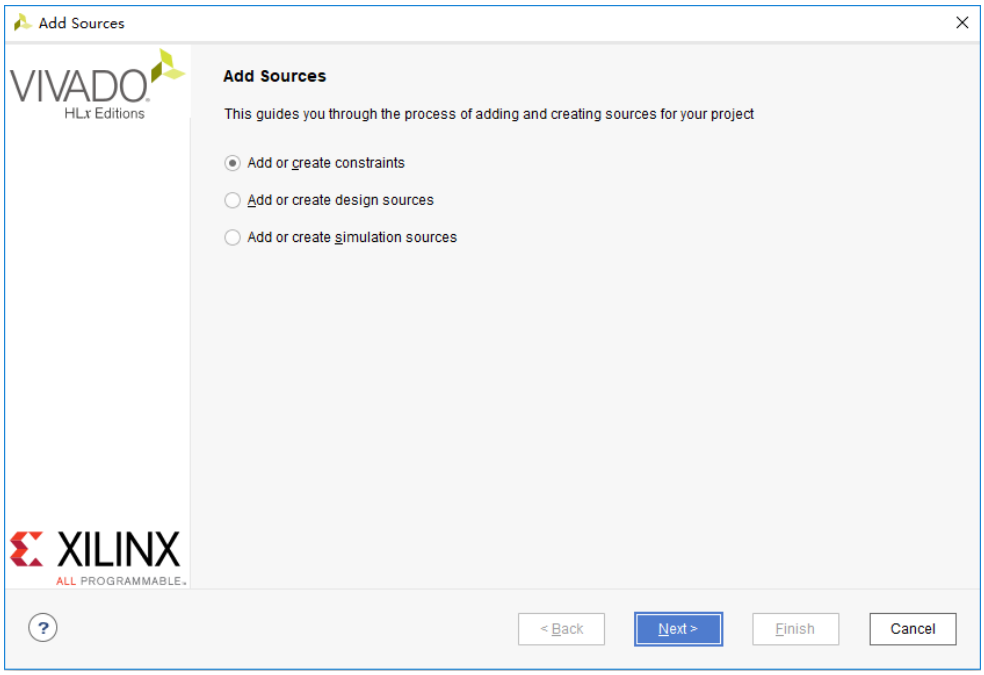
25) 弹出的对话框不作修改，点击“OK”， ，生成顶层文件



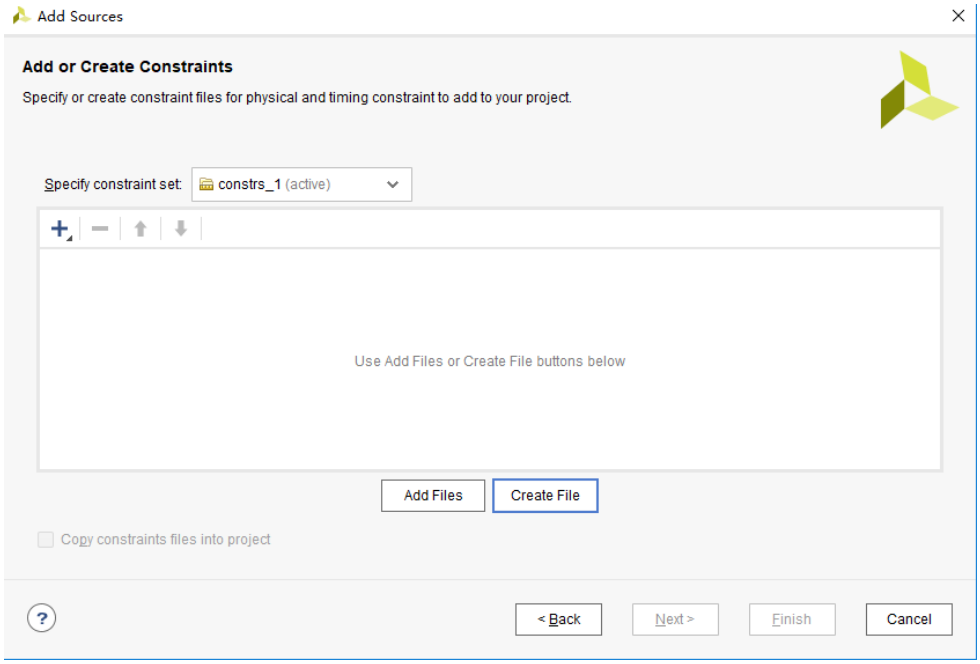
26) 接下来，我们需要为设计中的“ap_clk_0“及“led_V_0[0]”具体分配物理管脚。若当前没有处在“Sources”标签项下，则需要点击“Sources”，如图



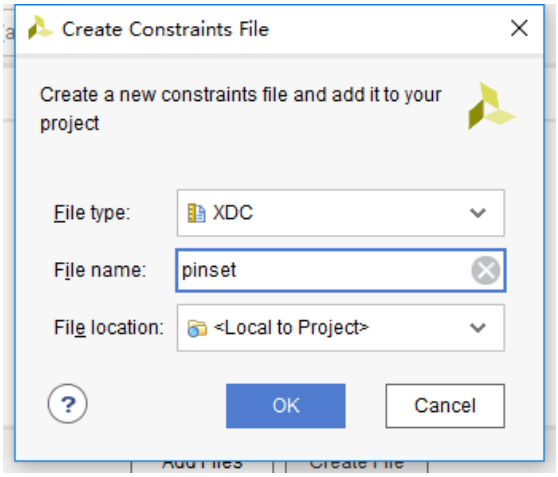
27) 右键图中”Constraints”，在菜单中选择“Add Sources…”，弹出向导。



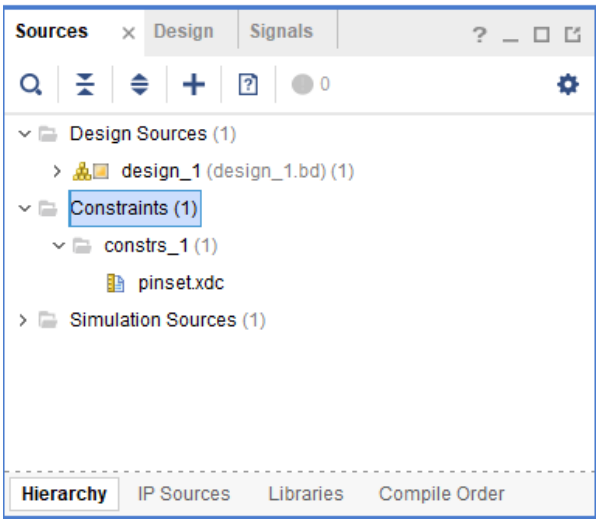
28) 不作任何改变，点击“Next”，在如下界面中，点击按钮”Create File”



29) 设置文件名称“pinset”，点击“OK “。



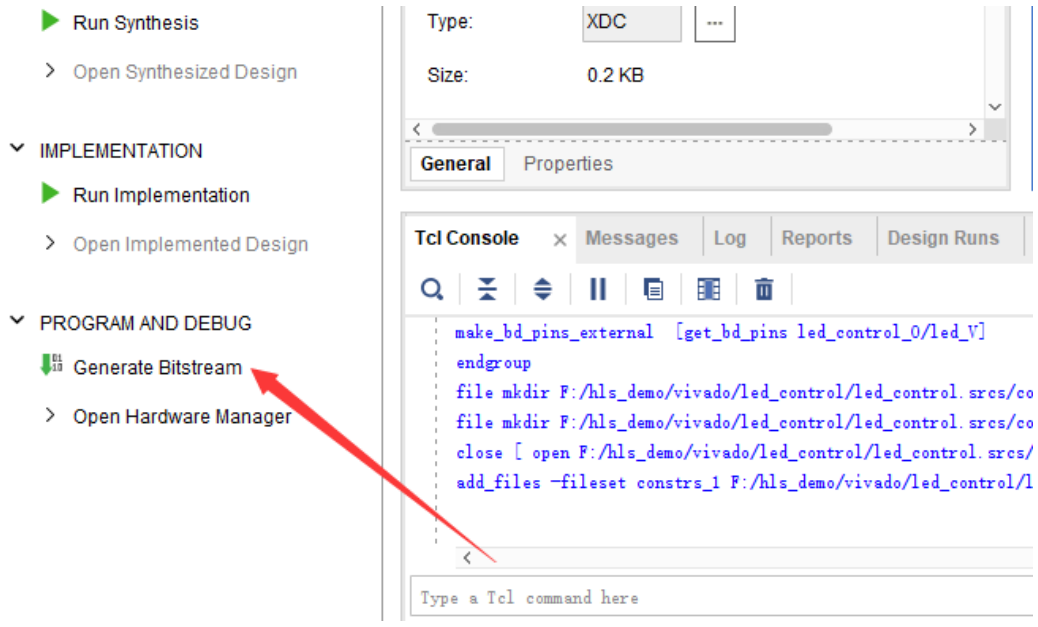
30) 回到前面的界面，点击“Finish”按钮，完成文件添加，此时我们展开“Constraints”，如下图。



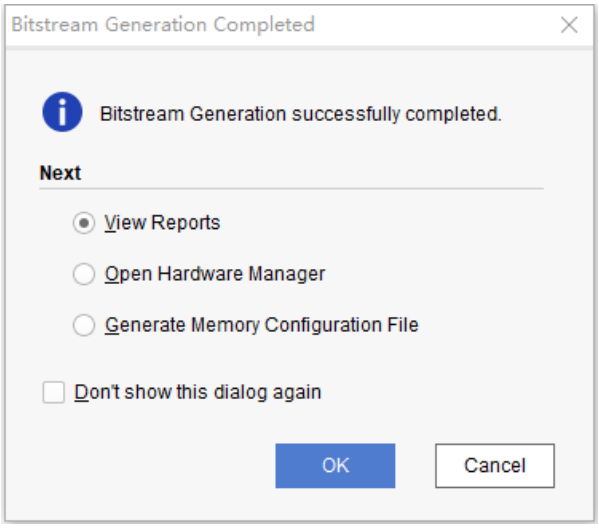
31) 双击”pinset.xdc”，在文件中输入如下内容，并“Ctrl+S”保存。注意：红色内容需要根据原理确定，必须替换。

```
set_property PACKAGE_PIN xxx    [get_ports ap_clk_0]
set_property PACKAGE_PIN xxx     [get_ports {led_V_0 [0]}]
set_property IOSTANDARD LVCMOSxx [get_ports {*}]
```

32) 至此，vivado 工程设计完毕，点击下述标注图标，生成 bit 文件。可能弹出保存文件对话框，点击“Save”即可。

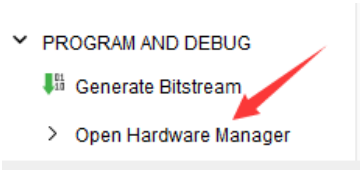


33) 生成文件可能需要几分钟，直至弹出如下对话框，点击“Cancel”按钮，bit 文件生成完成。

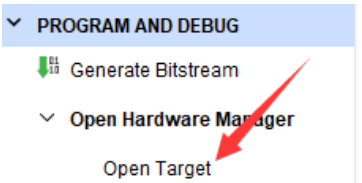


34) 接下来，可以加载刚生成的 bit 文件至板卡，先将板卡上电，并确定连接好 JTAG。

35) 点击 “Open Hardware Manager”

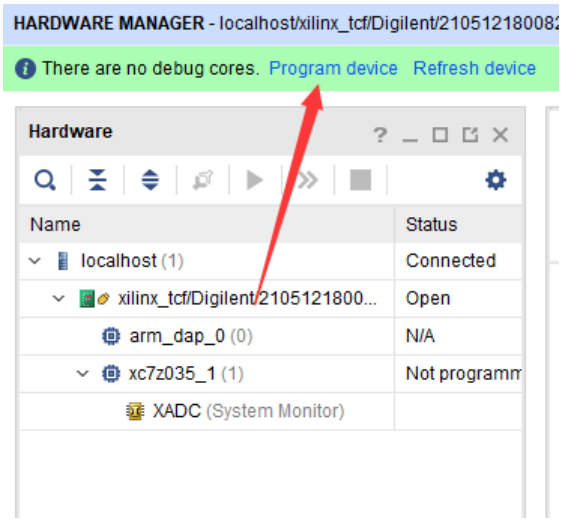


36) 点击“Open Target”→”Auto Connect”

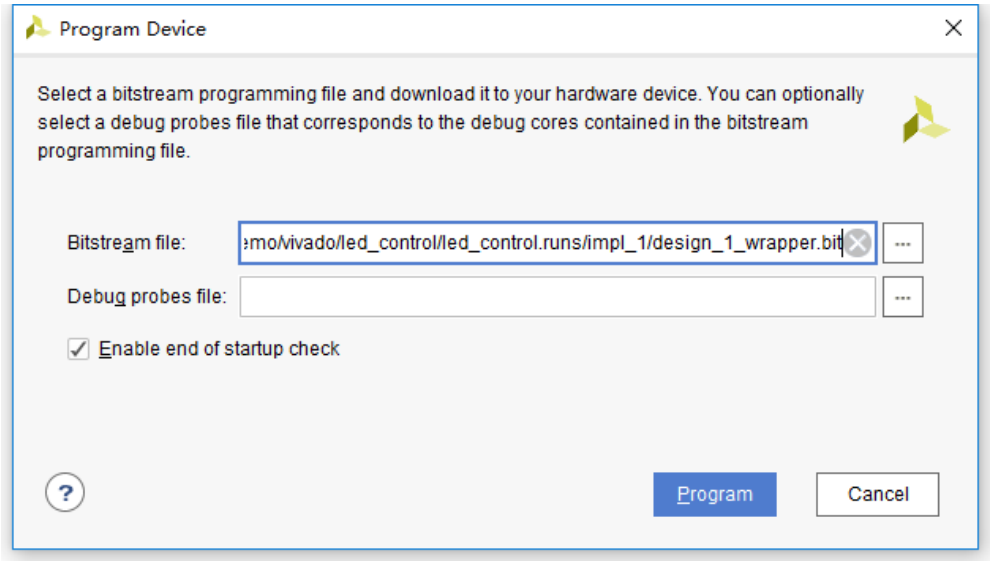




37) 点击”Program device“



38) 点击“Program“



39) 完成后，即可看到 led 灯以 1 秒的频率闪烁，至此，整个实验完成。

1.1.3 实验总结

通过 led 控制实验，我们学习了 HLS 与 vivado 的基本操作。对于 C、C++通过 HLS 转换为逻辑的流程，有了大致认识。

1.2 工程路径

名称	路径
Vivado 工程	vivado/led_control
HLS 工程	hls/led_control
bit 文件	/vivado/led_control/led_control.runs/impl_1/design_1_wrapper.bit

1.3 HLS 简介

Vivado hls 是 XILINX 推出的高层次综合工具，可以实现直接使用 C，C++ 以及 System C 语言规范对赛灵思可编程器件进行编程，无需手动创建 RTL，从而可加速 IP 创建。

通过在 C，C++中插入 HLS Pragmas 语句，定义我们设计的 IP 与外部的接口，优化综合结果，如减少执行周期、减少 FPGA 资源使用等。插入方法：在 HLS 软件界面，点击程序所在的文件，在右侧边栏有个 Directive，里面列出了程序中所有用到的变量，函数和循环结构，点击右键通过向导插入语句。或者我们手动输入。

1.3.1 Vivado HLS 包含库

任意精度的数据类型	整数和定点 (ap_cint.h, ap_int.h and systemc.h)
HLS 流	流数据结构模型 旨在实现最佳性能和面积 (hls_stream.h)
HLS Math	广泛支持标准 C (math.h) 和 C++ (cmath.h) 数学库的综合。支持浮点和定点功能：abs, atan, atanf, atan2, atan2, ceil, ceilf, copysign, copysignf, cos, cosf, coshf, expf, fabs, fabsf, floorf, fmax, fmin, logf, fpclassify, isfinite, isinf, isnan, isnormal, log, log10, modf, modff, recip, recipf, round, rsqrt, rsqrtf, 1/sqrt, signbit, sin, sincos, sincosf, sinf, sinhf, sqrt, tan, tanf, trunc
HLS 视频	视频库可使用 C++ 实现多个方面的建模视频设计，支持 视频功能、特定数据类型、存储器线路缓存以及存储器视窗 (hls_video.h)。 通过数据类型 hls::Mat， Vivado HLS 还与已有 OpenCV 功能兼容：AXIvideo2cvMat, AXIvideo2CvMat, AXIvideo2IplImage, cvMat2AXIvideo, CvMat2AXIvideo, cvMat2hlsMat, CvMat2hlsMat, CvMat2hlsWindow, hlsMat2cvMat, hlsMat2CvMat, hlsMat2IplImage, hlsWindow2CvMat, IplImage2AXIvideo, IplImage2hlsMat, AbsDiff, AddS, AddWeighted, And, Avg, AvgSdv, Cmp, CmpS, CornerHarris, CvtColor, Dilate, Duplicate, EqualizeHist, Erode, FASTX, Filter2D, GaussianBlur, Harris, HoughLines2, Integral, InitUndistortRectifyMap, Max, MaxS, Mean, Merge, Min, MinMaxLoc, MinS, Mul, Not, PaintMask, PyrDown, PyrUp, Range, Remap, Reduce, Resize, Set, Scale, Sobel, Split, SubRS, SubS, Sum, Threshold, Zero
HLS IP	集成 LogiCORE IP FFT 和 FIR Compiler (hls_fft.h, hls_fir.h, ap_shift_reg.h)
HLS 线性代数	cholesky, cholesky_inverse, matrix_multiply, qrf, qr_inverse, svd (hls_linear_algebra.h)
HLS DSP	atan2, awgn, cmpy, convolution_encoder, nco, qam_demod, qam_mod, sqrt, viterbi_decoder (hls_dsp.h)

1.3.2 Vivado HLS 接口

Argument Type	Scalar		Array			Pointer or Reference			HLS:: Stream
Interface Mode	Input	Return	I	I/O	O	I	I/O	O	I and O
ap_ctrl_none									
ap_ctrl_hs		D							
ap_ctrl_chain									
axis									
s_axilite									
m_axi									
ap_none	D					D			
ap_stable									
ap_ack									
ap_vld								D	
ap_ovld							D		
ap_hs									
ap_memory			D	D	D				
bram									
ap_fifo									D
ap_bus									

Supported D = Default Interface Not Supported

X14293

1.3.3 hls 官方教程

官方提供了两份很重要的教程：ug871-vivado-high-level-synthesis-tutorial.pdf 与 ug902-vivado-high-level-synthesis.pdf。里面有非常详细的使用说明。

第二章 状态指示 led

2.1 模块控制 block-level

回顾前面章节我们做的 led 实验，源代码中插入了“`#pragma HLS interface ap_ctrl_none port=return`”,它的作用是定义该 IP 无控制端口。通常，一个模块可能需要被外部控制停止、开始。也需要向控制方表明，本模块当前的状态：空闲、运行中、已完成、就绪。这就需要一个接口，hls 中，block-level I/O 协议为此定义了三种接口：ap_ctrl_none、 ap_ctrl_hs 和 ap_ctrl_chain。

我们分别来看，如果分别定义这三种接口，综合结果会怎么样

1. ap_ctrl_none

Interface

Summary

RTL Ports	Dir	Bits	Protocol	Source Object	C Type
ap_clk	in	1	ap_ctrl_none	led_control	return value
ap_rst	in	1	ap_ctrl_none	led_control	return value
led_V	out	1	ap_none	led_V	pointer

2. ap_ctrl_hs

Interface

Summary

RTL Ports	Dir	Bits	Protocol	Source Object	C Type
ap_clk	in	1	ap_ctrl_hs	led_control	return value
ap_rst	in	1	ap_ctrl_hs	led_control	return value
ap_start	in	1	ap_ctrl_hs	led_control	return value
ap_done	out	1	ap_ctrl_hs	led_control	return value
ap_idle	out	1	ap_ctrl_hs	led_control	return value
ap_ready	out	1	ap_ctrl_hs	led_control	return value
led_V	out	1	ap_none	led_V	pointer

3. ap_ctrl_chain

Interface

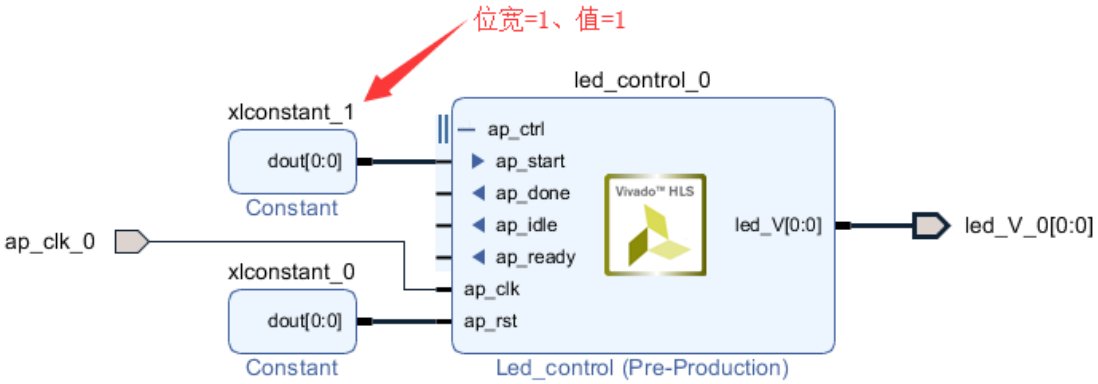
Summary

RTL Ports	Dir	Bits	Protocol	Source Object	C Type
ap_clk	in	1	ap_ctrl_chain	led_control	return value
ap_rst	in	1	ap_ctrl_chain	led_control	return value
ap_start	in	1	ap_ctrl_chain	led_control	return value
ap_done	out	1	ap_ctrl_chain	led_control	return value
ap_continue	in	1	ap_ctrl_chain	led_control	return value
ap_idle	out	1	ap_ctrl_chain	led_control	return value
ap_ready	out	1	ap_ctrl_chain	led_control	return value
led_V	out	1	ap_none	led_V	pointer

我们可以看到，ap_ctrl_none 无控制信号，ap_ctrl_hs 则多出了 start、done、idle、ready 信

号，ap_ctrl_chain 则在 ap_ctrl_hs 基础上多了 continue 信号。

hls 中， ap_ctrl_hs 为该接口的默认值。在使用中，需要将 start 置高，如图：



可以看出，选择默认 ap_ctrl_hs，使用起来比较麻烦，所以，之前的实验中，我们将其指定为 ap_ctrl_none。

2.2 可配置的模块

若 led 的闪烁频率可通过软件配置，就可以将其作为状态指示灯了。为此，我们修改 hls 代码如下：

```
#include <ap_int.h>

void led_register(ap_int<1> &led, int total_cnt, int high_cnt)
{
#pragma HLS INTERFACE ap_none port=led
#pragma HLS INTERFACE s_axilite port=total_cnt
#pragma HLS INTERFACE s_axilite port=high_cnt
#pragma HLS INTERFACE ap_ctrl_none port=return

    for(int i=0;i<total_cnt;i++)
    {
        led = (i<high_cnt)?1:0;
    }
}
```

这里，我们新增了两个参数 total_cnt、high_cnt，并将其指定为 s_axilite 接口，此时软件可通过 AXI4-Lite 接口来配置这两个参数值。

2.3 工程路径

名称	路径
Vivado 工程	vivado/led_register
HLS 工程	hls/ led_register
BOOT.bin 文件	vivado/led_register/led_register.sdk/app/bootimage/BOOT.bin

2.4 实验结果

程序运行后，可以看到 led 灯每秒闪烁一次。

第三章 浮点协处理

3.1 实验介绍

在应用中，有时会出现需要进行大量浮点或其它类型数据运算的情况，此时将占用大量 cpu 的时间。若将此类运算放至 fpga 端，cpu 仅需发起请求和读取结果，将节约大量 cpu 宝贵时间，特别在有多组相同类型操作需求的时候，我们利用 fpga 多个模块并发操作，将会大大缩短总时间，提高处理任务的实时性。

这里，我们设计一个模块，由 cpu 指定 float 类型数据存放地址和数据长度，求它们的平均值。计算完成后，通过中断方式，通知 cpu。

3.2 IP 创建

3.2.1 HLS 源代码

```
#include <string.h>

double sum;

void float_sum(float *src, int num)
{
     int len;
     float cachebuff[64];
#pragma HLS STREAM variable=cachebuff depth=64

     for(int i=0;i<num;i+=64)
     {
#pragma HLS loop_tripcount min=0 max=1024
#pragma HLS dataflow
          len = ((num-i) >= 64) ? 64 : num-i;
          memcpy(cachebuff, src+i, len*4);
          for(int j=0;j<len;j++)
          {
#pragma HLS pipeline
               sum += cachebuff[j];
          }
     }
}

float average_float(float *src, int num)
{
#pragma HLS INTERFACE m_axi depth=1024 offset=slave port=src
#pragma HLS INTERFACE s_axilite port=num
#pragma HLS INTERFACE s_axilite port=return

     sum = 0;
     float_sum(src, num);
     return (float)(sum/num);
}
```

3.2.2 接口介绍

这里共生成两个端口,axi4 和 axi4-lite。模块从传递进来的 src 地址处，通过 axi4 端口主动读取数据，数组长度 “num”与控制接口 “return”通过 axi4-lite 端口管理。

3.2.3 运算

for 循环内，dataflows 可以使 memcpy 与子 for 循环形成流水。插入“#pragma HLS pipeline “，作用是使本循环形成流水线，提高运算速度。

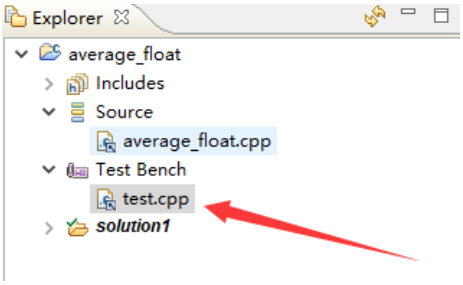
3.2.4 其它说明

“depth=1024 “与” #pragma HLS loop_tripcount min=0 max=1024”仅作用于 TestBench，前者表示当前数组最大长度为 1024，后者表示 for 循环最少循环 0 次，最多循环 1024 次。

3.3 TestBench

3.3.1 应用程序创建

根据需要，创建一个应用程序。如下图位置：



3.3.2 源代码

```
#include <stdio.h>

float average_float(float *src, int num);

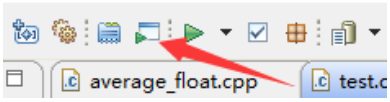
int main(void)
{
     float floatAry[1024];
     float average_local;
     float average_call;
     double sum;
     int num = (int)sizeof(floatAry)/sizeof(floatAry[0]);
     //初始化数据
     for(int i=0; i<num; i++)
     {
          floatAry[i] = 1.1f;
     }
     floatAry[101] += 0.1*1024;
     //软件计算平均值
     for(int i=0; i<num; i++)
     {
          sum += floatAry[i];
     }
     average_local = (float)sum/num;

     //调用ip顶层函数
     average_call = average_float(floatAry, num);
     if(average_local != average_call)
     {
          printf("app test fail, local=%f  call=%f!\n", average_local, average_call);
          return -1;
     }
```

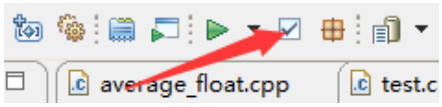
```
    printf("app test ok!\n");
    return 0;
}
```

3.3.3 C 仿真

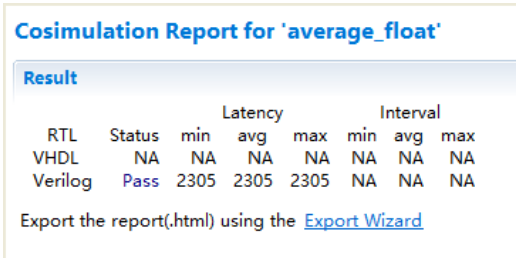
点击图示按钮，若验证通过，则会弹出包含“INFO: [SIM 1] CSim done with 0 errors.”语句的界面。



3.3.4 RTL 仿真



点击图示按钮，弹出的界面不作任何修改，点击“确定“按钮，软件运行完成后，弹出如下界面：



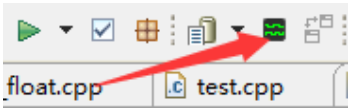
Cosimulation Report for 'average_float'

Result

		Latency			Interval		
RTL	Status	min	avg	max	min	avg	max
VHDL	NA	NA	NA	NA	NA	NA	NA
Verilog	Pass	2305	2305	2305	NA	NA	NA

Export the report(.html) using the Export Wizard

此时，我们发现，原本灰色的位置，变成如图所示图标，点击该图标，则可看到生成的波形图。

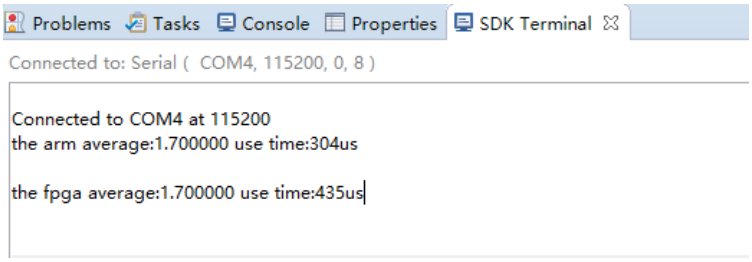


3.4 工程路径

名称	路径
Vivado 工程	vivado/average_float
HLS 工程	hls/average_float
BOOT.bin 文件	vivado/average_float/average_float.sdk/app/bootimage/BOOT.bin

3.5 运行结果

SDK 运行后，串口将打印出 arm 和 fpga 进行 1024 个浮点数求平均数用时。如图：


Problems
Tasks
Console
Properties
SDK Terminal
Connected to: Serial (COM4, 115200, 0, 8)
Connected to COM4 at 115200
the arm average:1.700000 use time:304us
the fpga average:1.700000 use time:435us

第四章 视频彩条

4.1 Vivado HLS 视频开发

4.1.1 与 OpenCV 关系

开源计算机视觉（OpenCV）被广泛用于开发计算机视觉应用，它包含 2500 多个优化的视频函数的函数库并且专门针对台式机处理器和 GPU 进行优化。OpenCV 的用户成千上万，OpenCV 的设计无需修改即可在 Zynq 器件的 ARM 处理器上运行。但是利用 OpenCV 实现的高清处理经常受外部存储器的限制，尤其是存储带宽会成为性能瓶颈，存储访问也会限制功耗效率。使用 VivadoHLS 高级语言综合工具，可以轻松实现 OpenCV C++视频处理设计到 RTL 代码的转换，输出硬件加速器或者直接在 FPGA 上实现实时视频处理功能。同时，Zynq All-programmable SOC 是实现嵌入式计算机视觉应用的极好方法，很好解决了在单一处理器上实现视频处理性能低功耗高的限制。

OpenCV 图像处理是基于存储器帧缓存而构建的，它总是假设视频 frame 数据存放在外部 DDR 存储器中，因此，OpenCV 对于访问局部图像性能较差，因为处理器的小容量高速缓存性能不足以完成这个任务。而且出于性能考虑，基于 OpenCV 设计的架构比较复杂，功耗更高。在对分辨率或帧速率要求低，或者在更大的图像中对需要的特征或区域进行处理是，OpenCV 似乎足以满足很多应用的要求，但对于高分辨率高帧率实时处理的场景下，OpenCV 很难满足高性能和低功耗的需求。

基于视频流的架构能提供高性能和低功耗，链条化的图像处理函数能减少外部存储器访问，针对视频优化的行缓存和窗口缓存比处理器高速缓存更简单，更易于用 FPGA 部件，使用 VivadoHLS 中的数据流优化来实现.

VivadoHLS 对 OpenCV 的支持，不是指可以将 OpenCV 的函数库直接综合成 RTL 代码，而是需要将代码转换为可综合的代码，这些可综合的视频库称为 HLS 视频库，由 VivadoHLS 提供。

OpenCV 函数不能直接通过 HLS 进行综合，因为 OpenCV 函数一般都包含动态的内存分配、浮点以及假设图像在外部存储器中存放或者修改。

VivadoHLS 视频库用于替换很多基本的 OpenCV 函数，它与 OpenCV 具有相似的接口和算法，主要针对在 FPGA 架构中实现的图像处理函数，包含了专门面向 FPGA 的优化，比如定点运算而非浮点运算（不必精确到比特位），片上的行缓存（line buffer）和窗口缓存(window buffer)。

4.1.2 VivadoHLS 视频库函数

HLS 视频库是包含在 hls 命名空间内的 C++代码。使用时需要包含头文件“hls_video.h”。

该库与 OpenCV 等具有相似的接口和等效的行为，例如：

OpenCV 库：cvScale(src, dst, scale, shift);

HLS 视频库：hls:Scale<...>(src, dst, scale, shift);

一些构造函数具有类似的或替代性的模板参数，例如：

OpenCV 库：cv::Mat mat(rows, cols, CV_8UC3);

HLS 视频库：hls::Mat mat(rows, cols);

4.2 实验介绍

在图像处理中，我们常用彩条(colorbar)来验证视频输出接口，或作为无视频时的输出方案。我们这里的 colorbar，为 8 个不同颜色的竖条，中间三分之一的竖条，从左至右滚动，其它区域静止。且输出的图像，可以动态修改分辨率。

4.3 HLS IP 创建

4.3.1 源代码

```
#include <hls_stream.h>
#include <hls_video.h>
#include <ap_axi_sdata.h>

typedef ap_axiu<24,1,1,1> pixel_data;
typedef hls::stream<pixel_data> pixel_stream;

template<int ROWS, int COLS, int T>
void createColorBar(hls::Mat<ROWS, COLS, T> &imgColorbar)
{
     hls::Scalar<HLS_MAT_CN(T), HLS_TNAME(T)> pixel;
     int move, wBar;
     int rows = imgColorbar.rows;
     int cols = imgColorbar.cols;
     static int moving = 0;

     for(int row=0;row<rows;row++)
     {
#pragma HLS loop_flatten off
          if( (row < rows/3) || (row >= rows/3*2) )
          {
               move = 0;
          }
          else
          {
               move = moving;
          }
          wBar = cols>>3;
          for(int col=0;col<cols;col++)
          {
#pragma HLS pipeline
               int index = ((col+move)/wBar)&0x7;
               switch(index)
               {
               case 0:
                    pixel.val[0] = 0xff;
                    pixel.val[1] = 0xff;
                    pixel.val[2] = 0xff; //white
                    break;
               case 1:
                    pixel.val[0] = 0x00;
                    pixel.val[1] = 0xff;
                    pixel.val[2] = 0xff; //yellow
                    break;
               case 2:
                    pixel.val[0] = 0xff;
                    pixel.val[1] = 0xff;
                    pixel.val[2] = 0x00; //cyan
                    break;
               case 3:
```

```
                    pixel.val[0] = 0x00;
                    pixel.val[1] = 0xff;
                    pixel.val[2] = 0x00; //green
                    break;
               case 4:
                    pixel.val[0] = 0xff;
                    pixel.val[1] = 0x00;
                    pixel.val[2] = 0xff; //magenta
                    break;
               case 5:
                    pixel.val[0] = 0x0;
                    pixel.val[1] = 0x0;
                    pixel.val[2] = 0xff; //red
                    break;
               case 6:
                    pixel.val[0] = 0xff;
                    pixel.val[1] = 0x00;
                    pixel.val[2] = 0x00; //blue
                    break;
               default:
                    pixel.val[0] = 0x00;
                    pixel.val[1] = 0x00;
                    pixel.val[2] = 0x00; //black
                    break;
               }

               imgColorbar << pixel;
          }
     }
     if(!moving)
     {
          moving = cols;
     }
     else
     {
          moving--;
     }
}

void colorbar(pixel_stream &dst, int rows, int cols)
{
#pragma HLS INTERFACE axis port=dst
#pragma HLS INTERFACE s_axilite port=rows
#pragma HLS INTERFACE s_axilite port=cols
#pragma HLS INTERFACE s_axilite port=return
#pragma HLS INTERFACE ap_ctrl_none port=return

     hls::Mat<1080, 1920, HLS_8UC3> imgColorbar(rows, cols);

#pragma HLS dataflow
     createColorBar(imgColorbar);
     hls::Mat2AXIvideo(imgColorbar, dst);
}
```

4.3.2 接口介绍

数据类型 pixel_stream 的定义为 hls::stream< ap_axiu<24,1,1,1> > 。这是 HLS 的流接口，其中 24 指 RGB 数据位宽共 24Bit，其它位为流控制信号。通常，各模块之间视频数据接口都使用流接口。这里，我们将“dst”定义为视频输出流。

“rows”与 “cols”分别定义输出图像的高与宽，”mode”用于表明当前是否处于配置状态，若处于配置状态，则不输出图像。这里，我们使用 axi-lite 来管理这些接口，包括模块的控制接口。

4.3.3 hls::Mat 介绍

OpenCV 中常见的与图像操作有关的数据容器有 Mat，CvMat 和 IplImage，这三种类型都可以代表和显示图像，但是，Mat 类型侧重于计算，数学性较高，openCV 对 Mat 类型的计算也进行了优化。而 CvMat 和 IplImage 类型更侧重于“图像”，opencv 对其中的图像操作（缩放、单通道提取、图像阈值操作等）进行了优化。M at 类型较 CvMat 与 IplImage 类型来说，有更强的矩阵运算能力，支持常见的矩阵运算。在计算密集型的应用当中，将 CvMat 与 IplImage 类型转化为 Mat 类型将大大减少计算时间花费。

VivadoHLS 视频处理函数库使用 hls::Mat<>数据类型，这种类型用于模型化视频像素流处理，在 HLS 实现 OpenCV 的设计中，需要将输入和输出 HLS 可综合的视频设计接口，修改为 Video stream 接口，也就是采用 HLS 提供的 video 接口可综合函数，实现 AXI4 video stream 到 VivadoHLS 中 hls::Mat<>类型间的转换。

在本例程中，我们在顶层函数中定义了 hls::Mat<1080, 1920, HLS_8UC3> imgColorbar(rows, cols)。其中，前面的尖括号内分别表示我们的图像最大支持分辨率为 1920x1080，bit24 的图像，后面括号参数表示当前实际的高与宽分别为 rows、cols。

4.3.4 优化

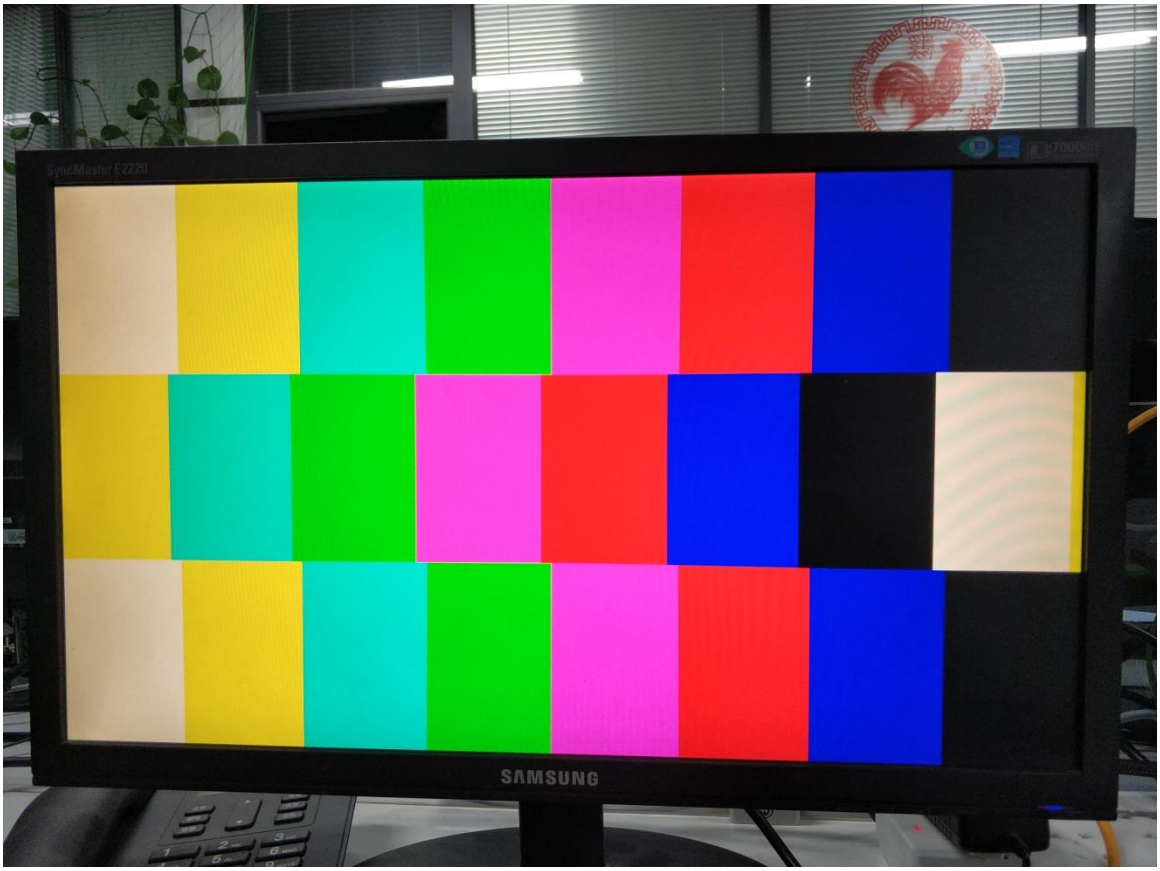
在顶层函数 colorbar 中，我们使用了语句” #pragma HLS dataflow”，以使能够在下面的函数中，形成流水操作，提高数据处理效率。

在 createColorBar 中，在第一个 for 循环内嵌入” #pragma HLS loop_flatten off”，表明放弃与内部 for 循环整合，这样会增加时钟周期，但改善了信号延迟。

4.4 工程路径

名称	路径
Vivado 工程	vivado/colorbar
HLS 工程	hls/colorbar
BOOT.bin 文件	vivado/colorbar/colorbar.sdk/app/bootimage/BOOT.bin

4.5 实验结果



显示一个 colorbar，中间部分滚动显示。

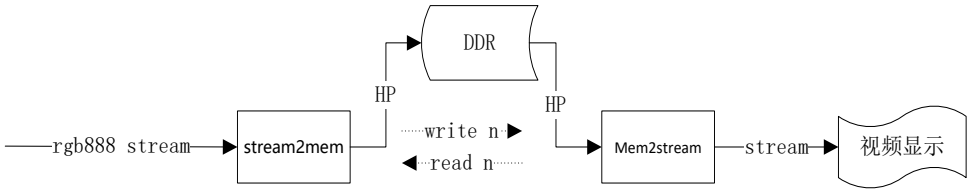
需要注意，显示输出分辨率在不断变化，所以画面会隔一段时间黑掉，属于正常现象。

第五章 视频帧缓存读写管理

5.1 实验介绍

首先，将镜头的图像(转换为 RGB888)存储至 DDR，然后通过读取 DDR 来显示视频图像。这里需要解决图像刷新速率不一致时，可能造成的图像割裂状问题。

这里共设计了二个 HLS 模块：stream2mem 模块，mem2stream 模块，流程图如下。



这里需要解决的一个问题是，若写 DDR 与读 DDR 速度不匹配，则会造成图像显示时裂开的现像。解决的方法是：DDR 使用三帧缓存，HLS 读模块与写模块分别向对方反馈帧索引号。

5.2 模块主要代码

```
template<int ROWS, int COLS, int T>
void mat2mem(hls::Mat<ROWS, COLS, T> &img, unsigned int *pMemPort, int &w, int r)
{
      hls::Scalar<HLS_MAT_CN(T), HLS_TNAME(T)> pixelA, pixelB, pixelC, pixelD;
      unsigned int cacheBuff[COLS/4];
      int local_rows = img.rows;
      int local_cols = img.cols;
      int line_len   = img.cols*3/4;
      static int index=0;
      int n = (index<2)?(index+1):0;
      index = (n == r)?index:n;
      w = index;
#pragma HLS STREAM variable=cacheBuff depth=COLS/4

      for(int row=0; row<local_rows; row++)
      {
#pragma HLS loop_tripcount min=12 max=1080
#pragma HLS dataflow
            for(int col=0; col<local_cols; col+=4)
            {
#pragma HLS loop_tripcount min=3 max=480
                  img >> pixelA;
                  img >> pixelB;
                  cacheBuff[0] = (pixelA.val[0]<<0)|(pixelA.val[1]<<8)|(pixelA.val[2]<<16)|(pixelB.val[0]<<24);
                  img >> pixelC;
                  cacheBuff[1] = (pixelB.val[1]<<0)|(pixelB.val[2]<<8)|(pixelC.val[0]<<16)|(pixelC.val[1]<<24);
                  img >> pixelD;
                  cacheBuff[2] = (pixelC.val[2]<<0)|(pixelD.val[0]<<8)|(pixelD.val[1]<<16)|(pixelD.val[2]<<24);
            }
            writemem(pMemPort, (baseAddr[index]>>2)+(row*line_len), cacheBuff, line_len*4);
      }
}
```

在这里，我们将数组 cacheBuff 定义为流类型，方便我们排流水。另外，baseAddr 为全局变量，但我们仍然可以将其定义为模块的外部接口，而不必是 top 函数的参数。

内存读取与写入时，调用系统函数 memcpy。若直接引用单个数组成员，将严重浪费 DDR 带宽，不推荐。

5.1 工程路径

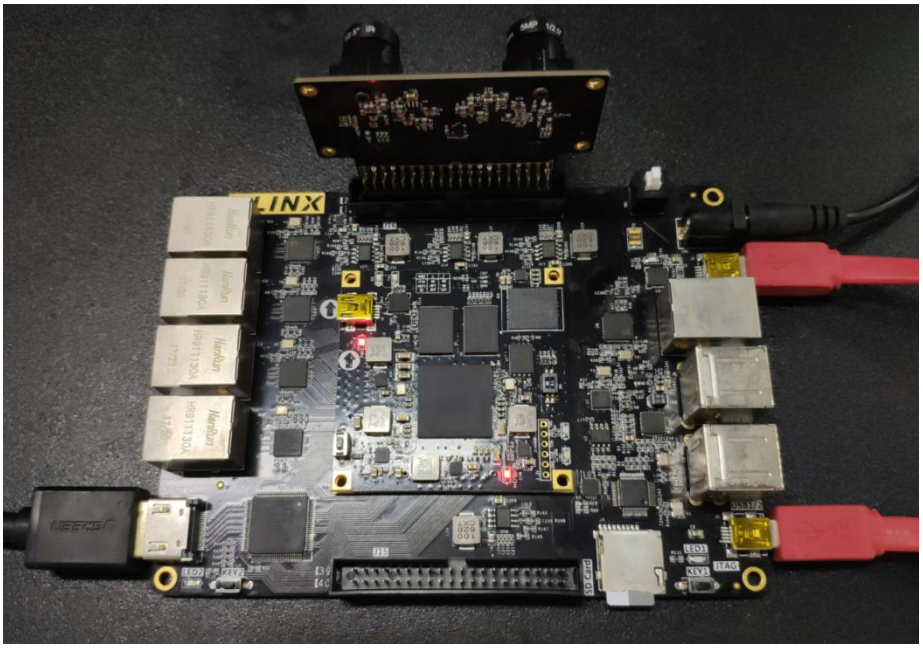
名称	路径
Vivado 工程	vivado/video_show
HLS 工程	hls/mem2stream
HLS 工程	hls/stream2mem
BOOT.bin 文件	vivado/video_show/video_show.sdk/app/bootimage/BOOT.bin

5.2 实验结果

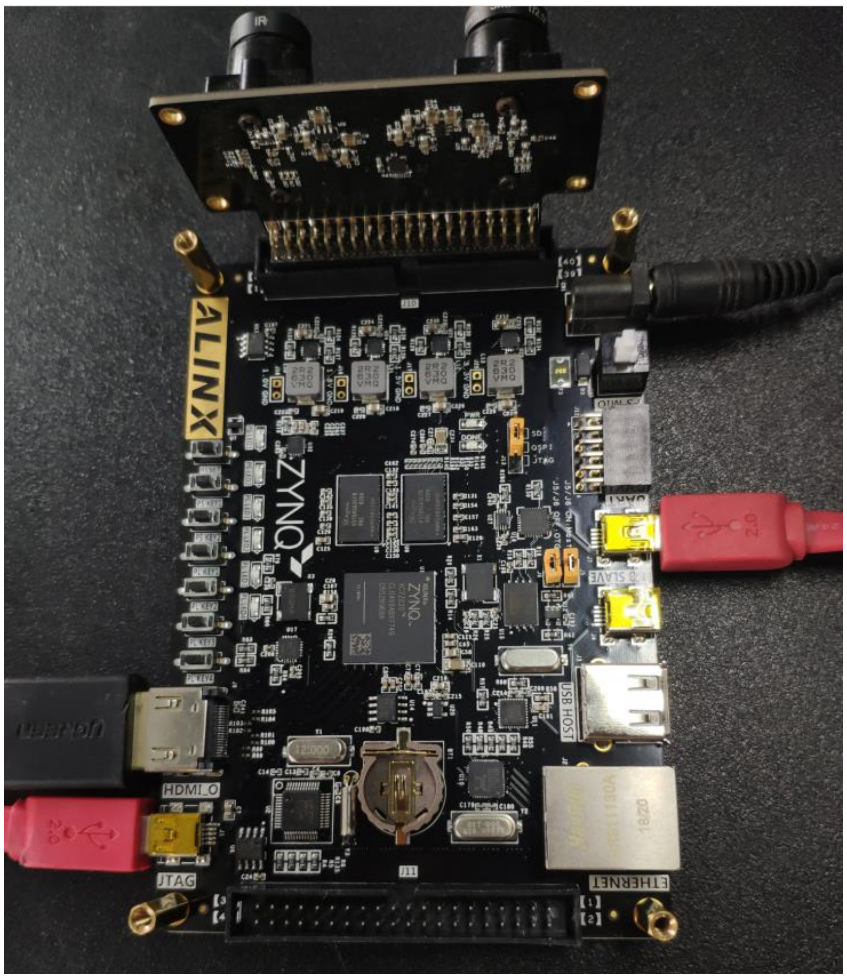


显示双目视频中的一个视频图像。当视频中出现快速移动物体时，不会有割裂状。当前视频在几个分辨率之间不断切换。低分辨率时，仅能显示部分图像。

需要注意，显示输出分辨率在不断变化，所以画面会隔一段时间黑掉，属于正常现象。



AX7021 硬件连接图（J16 扩展口）

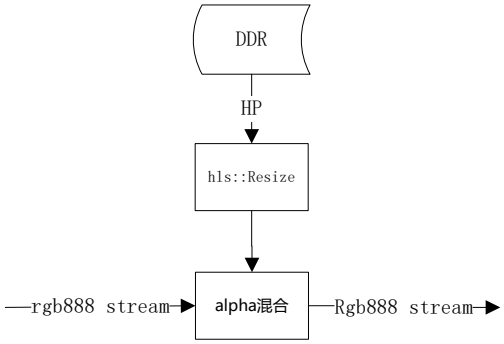


AX7020 硬件连接图(J10 扩展口)

第六章 图像缩放叠加

6.1 实验介绍

接收到 RGB888 的视频流后，读取一片 DDR 数据，并将其中数据进行放大或者缩小操作，然后与流 alpha 混合，并输出，如图：



这样设计的好处是，模块可以任意多个级联，从而实现多位置多层次的图像叠加。

需要注意的问题是，若多模块读取 DDR，则需要注意 HP 的带宽，特别是在图像垂直方向缩小的时候，需要更高的带宽。

6.2 模块主要代码

```
template<int ROWS, int COLS, int T>
void overlyOnMat(hls::Mat<ROWS, COLS, T> &img_bottom, hls::Mat<ROWS, COLS, T> &img_top, hls::Mat<ROWS, COLS, T> &img_out)
{
	hls::Scalar<HLS_MAT_CN(T), HLS_TNAME(T)> pixelBottom;
	hls::Scalar<HLS_MAT_CN(T), HLS_TNAME(T)> pixelTop;
	hls::Scalar<HLS_MAT_CN(T), HLS_TNAME(T)> pixelMix;
	int local_rows = img_out.rows;
	int local_cols = img_out.cols;
	unsigned int x = overly_x;
	unsigned int y = overly_y;
	unsigned int h = img_top.rows;
	unsigned int w = img_top.cols;
	for(int row=0; row<local_rows; row++)
	{
#pragma HLS loop_tripcount min=12 max=1080
		for(int col=0; col<local_cols; col++)
		{
#pragma HLS loop_tripcount min=3 max=1920
#pragma HLS pipeline
			img_bottom >> pixelBottom;
			if(((y+h)<=local_rows) && ((x+w)<=local_cols) && h && w && (row >= y) && (row < (y+h)) && (col >= x) && (col <(x+w)))
			{
				img_top >> pixelTop;
				pixelMix.val[0] = ((255-overly_alpha)*pixelBottom.val[0]+overly_alpha*pixelTop.val[0])/255;
				pixelMix.val[1] = ((255-overly_alpha)*pixelBottom.val[1]+overly_alpha*pixelTop.val[1])/255;
				pixelMix.val[2] = ((255-overly_alpha)*pixelBottom.val[2]+overly_alpha*pixelTop.val[2])/255;
			}
			else
			{
				pixelMix.val[0] = pixelBottom.val[0];
				pixelMix.val[1] = pixelBottom.val[1];
				pixelMix.val[2] = pixelBottom.val[2];
			}
			img_out << pixelMix;
		}
	}
}

template<int ROWS, int COLS, int T>
void createImgCoverlay(hls::Mat<ROWS, COLS, T> &img, unsigned int *pMem, int cols)
{
	int len = img.cols*3;
	unsigned int linebuff[COLS];
	unsigned int d1, d2, d3;
```

```
        hls::Scalar<HLS_MAT_CN(T), HLS_TNAME(T)> pixel;
        int local_rows = img.rows;
        int local_cols = img.cols;
#pragma HLS STREAM variable=linebuff depth=COLS

        for(int row=0; row<local_rows; row++)
        {
#pragma HLS loop_tripcount min=12 max=1080
#pragma HLS dataflow
            memcpy(linebuff, pMem+(((zoom_y+row)*cols+zoom_x)*3>>2), len);
            for(int col=0; col<local_cols; col+=4)
            {
#pragma HLS loop_tripcount min=12 max=480
                d1 = linebuff[col*3+0];
                pixel.val[0] = d1 >> 0;
                pixel.val[1] = d1 >> 8;
                pixel.val[2] = d1 >> 16;
                img << pixel;
                d2 = linebuff[col*3+1];
                pixel.val[0] = d1 >> 24;
                pixel.val[1] = d2 >> 0;
                pixel.val[2] = d2 >> 8;
                img << pixel;
                d3 = linebuff[col*3+2];
                pixel.val[0] = d2 >> 16;
                pixel.val[1] = d2 >> 24;
                pixel.val[2] = d3 >> 0;
                img << pixel;
                pixel.val[0] = d3 >> 8;
                pixel.val[1] = d3 >> 16;
                pixel.val[2] = d3 >> 24;
                img << pixel;
            }
        }
}

void overlaystream(vstream_t &video_in, vstream_t &video_out, unsigned int *pMem, int rows, int cols)
{
#pragma HLS INTERFACE axis port=video_in
#pragma HLS INTERFACE axis port=video_out
#pragma HLS INTERFACE m_axi port=pMem offset=slave
#pragma HLS INTERFACE s_axilite port=rows
#pragma HLS INTERFACE s_axilite port=cols
#pragma HLS INTERFACE s_axilite port=return
#pragma HLS INTERFACE s_axilite port=overly_x
#pragma HLS INTERFACE s_axilite port=overly_y
#pragma HLS INTERFACE s_axilite port=overly_w
#pragma HLS INTERFACE s_axilite port=overly_h
#pragma HLS INTERFACE s_axilite port=overly_alpha
#pragma HLS INTERFACE s_axilite port=zoom_x
#pragma HLS INTERFACE s_axilite port=zoom_y
#pragma HLS INTERFACE s_axilite port=zoom_w
#pragma HLS INTERFACE s_axilite port=zoom_h

        hls::Mat<IMG_MAX_ROWS, IMG_MAX_COLS, HLS_8UC3> img_in(rows, cols);
        hls::Mat<IMG_MAX_ROWS, IMG_MAX_COLS, HLS_8UC3> img_out(rows, cols);
        hls::Mat<IMG_MAX_ROWS, IMG_MAX_COLS, HLS_8UC3> img_coverlay(zoom_h, zoom_w);
        hls::Mat<IMG_MAX_ROWS, IMG_MAX_COLS, HLS_8UC3> img_coverlay_resize(overly_h, overly_w);

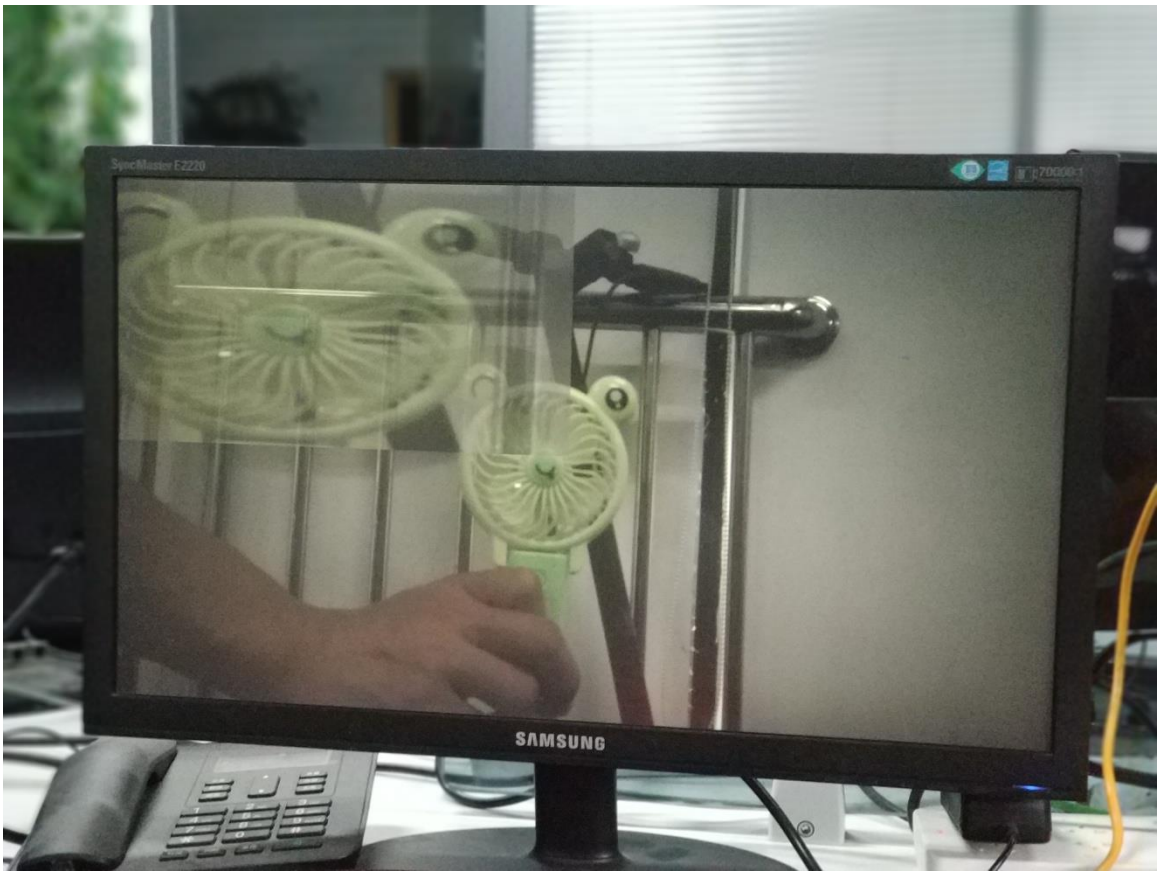
#pragma HLS dataflow

        hls::AXIvideo2Mat(video_in, img_in);
        createImgCoverlay(img_coverlay, pMem, cols);
        hls::Resize(img_coverlay, img_coverlay_resize);
        overlyOnMat(img_in, img_coverlay_resize, img_out);
        hls::Mat2AXIvideo(img_out, video_out);
}
```

6.3 工程路径

名称	路径
Vivado 工程	vivado/video_on_video
HLS 工程	hls/overlaystream
HLS 工程	hls/mem2stream
HLS 工程	hls/stream2mem
BOOT.bin 文件	vivado/video_on_video/video_on_video.sdk/video/bootimage/BOOT.bin

6.4 实验结果



图中，将视频中心部分放大，与原图 alpha 混合叠加在左上角。

第七章 字符叠加

7.1 实验介绍

字符叠加与图像叠加类似，仅数据格式不同，字符数据使用 argb8888 格式。字符点阵信息由 SDK 处理。

7.2 模块主要代码

```
template<int ROWS, int COLS, int T>
void overlyOnMat(hls::Mat<ROWS, COLS, T> &img_in, hls::Mat<ROWS, COLS, T> &img_out, unsigned int *pMem)
{
	hls::Scalar<HLS_MAT_CN(T), HLS_TNAME(T)> pixel;
	unsigned int cacheBuff[COLS];
	int local_rows = img_in.rows;
	int local_cols = img_in.cols;
	unsigned int x = overly_x;
	unsigned int y = overly_y;
	unsigned int w = overly_w;
	unsigned int h = overly_h;
	unsigned int d;
	unsigned char top_a, top_r, top_g, top_b, back_r, back_g, back_b;
	unsigned short acc_r, acc_g, acc_b;
#pragma HLS STREAM variable=cacheBuff depth=COLS*2
	for(int row=0; row<local_rows; row++)
	{
#pragma HLS loop_tripcount min=12 max=1080
#pragma HLS dataflow
		readmem(pMem, cacheBuff, local_rows, local_cols, row, x, y, w, h);
		for(int col=0; col<local_cols; col++)
		{
#pragma HLS loop_tripcount min=3 max=1920
#pragma HLS pipeline
			img_in >> pixel;
			if(((y+h)<=local_rows) && ((x+w)<=local_cols) && h && w && (row >= y) && (row < (y+h)) && (col >= x) && (col <(x+w)))
			{
				d = cacheBuff[col-x];
				top_r = d>>0;
				top_g = d>>8;
				top_b = d>>16;
				top_a = d>>24;
				back_r = pixel.val[0];
				back_g = pixel.val[1];
				back_b = pixel.val[2];
				acc_r = (255-top_a)*back_r+top_a*top_r;
				acc_g = (255-top_a)*back_g+top_a*top_g;
				acc_b = (255-top_a)*back_b+top_a*top_b;
				pixel.val[0] = acc_r/255;
				pixel.val[1] = acc_g/255;
				pixel.val[2] = acc_b/255;
			}
			img_out << pixel;
		}
	}
}

void overlayosd(vstream_t &video_in, vstream_t &video_out, unsigned int *pMem, int rows, int cols)
{
#pragma HLS INTERFACE axis port=video_in
#pragma HLS INTERFACE axis port=video_out
#pragma HLS INTERFACE m_axi port=pMem offset=slave
#pragma HLS INTERFACE s_axilite port=rows
#pragma HLS INTERFACE s_axilite port=cols
#pragma HLS INTERFACE s_axilite port=return
#pragma HLS INTERFACE s_axilite port=overly_x
#pragma HLS INTERFACE s_axilite port=overly_y
#pragma HLS INTERFACE s_axilite port=overly_w
#pragma HLS INTERFACE s_axilite port=overly_h

	hls::Mat<IMG_MAX_ROWS, IMG_MAX_COLS, HLS_8UC3> img_in(rows, cols);
	hls::Mat<IMG_MAX_ROWS, IMG_MAX_COLS, HLS_8UC3> img_out(rows, cols);

#pragma HLS dataflow

	hls::AXIvideo2Mat(video_in, img_in);
	overlyOnMat(img_in, img_out, pMem);
	hls::Mat2AXIvideo(img_out, video_out);
}
```

7.3 工程路径

名称	路径
Vivado 工程	vivado/video_osd
HLS 工程	hls/overlayosd
HLS 工程	hls/mem2stream
HLS 工程	hls/stream2mem
BOOT.bin 文件	vivado/video_osd/video_osd.sdk/app/bootimage/BOOT.bin

7.4 实验结果



图中，左上角叠加了一个图标，视频正中心叠加了“芯驿电子，字符叠加测试”。

第八章 图像对比度调整

8.1 实验介绍

直方图均衡技术将原始图像的灰度直方图从比较集中的某个灰度区间变成在全部灰度范围内的均匀分布，是一种常用的图像增强方法。

首先,我们需要将 RGB888 图像转化为 YCrCb,统计当前 Y 分量直方图分布(应用于下一帧),并通过上一帧的统计数据均衡化当前 Y 分量，再转换回 RGB888 并输出。

8.2 模块主要代码

```
template<int SRC_T, int DST_T,int ROW, int COL, int N>
void dualAryEqualize(
          hls::Mat<ROW, COL, SRC_T> &_src,
          hls::Mat<ROW, COL, DST_T> &_dst,
          int filter)
{
     static ap_uint<8> map[N];
     ap_uint<BitWidth<ROW*COL>::Value> hist_out1[N];
     ap_uint<BitWidth<ROW*COL>::Value> hist_out2[N];
     hls::Scalar<HLS_MAT_CN(SRC_T), HLS_TNAME(SRC_T)> tempsrc1, tempsrc2;
     int pos1, pos2;

#pragma HLS ARRAY_PARTITION variable=hist_out1 complete dim=1
#pragma HLS ARRAY_PARTITION variable=hist_out2 complete dim=1
     for(int i=0;i<N;i++)
     {
          hist_out1[i]=0;
          hist_out2[i]=0;
     }

     for(ap_uint<13> row=0; row < ROW; row++)
     {
#pragma HLS LOOP_TRIPCOUNT min=ROW max=ROW
          for(ap_uint<13> col=0; col < COL ; col = col+2)
          {
#pragma HLS PIPELINE
#pragma HLS LOOP_FLATTEN OFF
#pragma HLS LOOP_TRIPCOUNT min=2 max=2
               _src >> tempsrc1;
               _src >> tempsrc2;
               pos1 = tempsrc1.val[0];
               pos2 = tempsrc2.val[0];
               hist_out1[pos1] += 1;
               hist_out2[pos2] += 1;
               tempsrc1.val[0] = map[pos1];
               tempsrc2.val[0] = map[pos2];
               _dst << tempsrc1;
               _dst << tempsrc2;
          }
     }
     int count_total = 0, tmp_hist;
     float scale;
     for(int i=0;i<N;i++)
     {
#pragma HLS PIPELINE
          tmp_hist = hist_out1[i]+hist_out2[i];
          count_total += tmp_hist;
          scale = count_total*255/ROW/COL;
          scale = (scale-i)*0.01*filter + i;
          if(scale < 16)scale = 16;
          if(scale > 255)scale = 235;
          map[i] = hls::sr_cast< ap_uint<8> > (scale);
     }
}
```

这里进行数据统计的时候，我们一次处理了两个像素点。因为在一个时钟内，我们无法完成“hist_out1[pos1] += 1;”，即读一个 ram 内容，将其加一并写回。但可以在两个时钟周期，完成一个读写操作

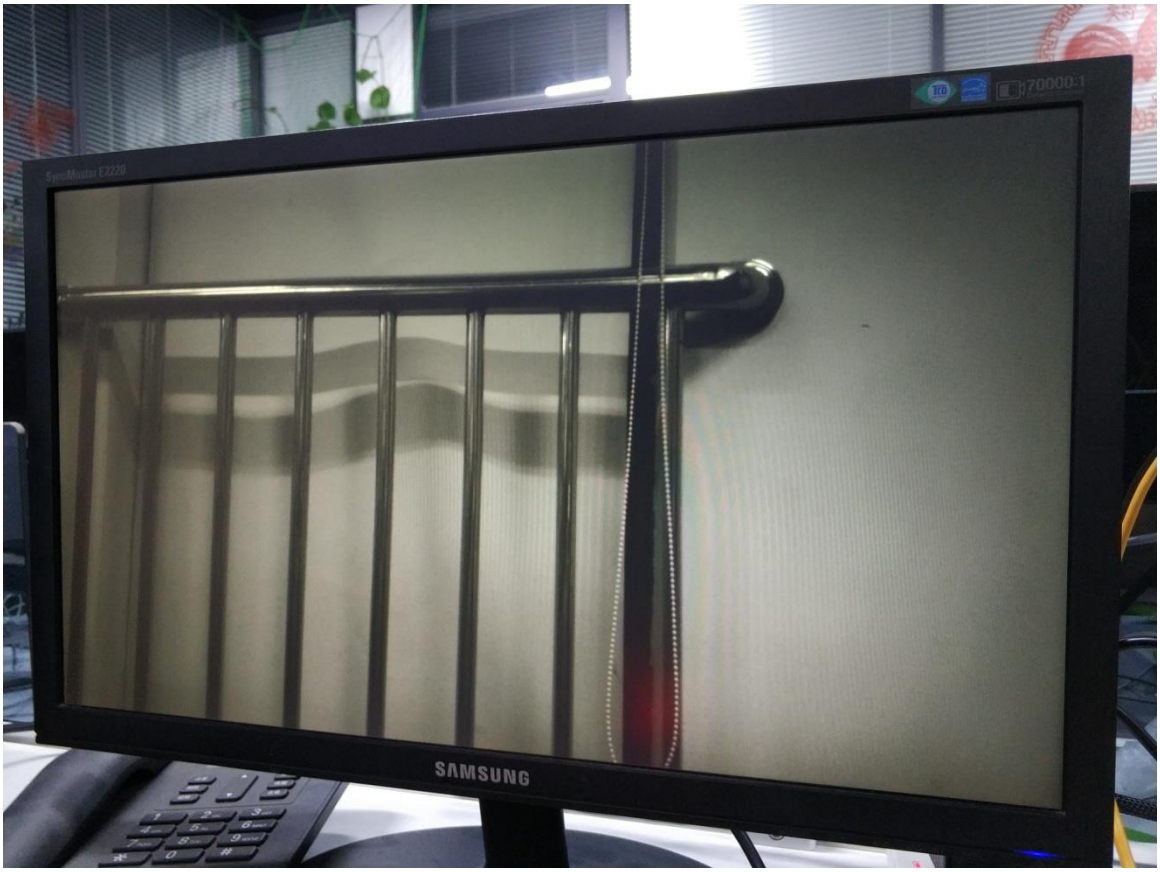
8.3 工程路径

名称	路径
Vivado 工程	vivado/contrast_adj
HLS 工程	hls/contrastadj
HLS 工程	hls/mem2stream
HLS 工程	hls/stream2mem
BOOT.bin 文件	vivado/contrast_adj/contrast_adj.sdk/app/bootimage/BOOT.bin

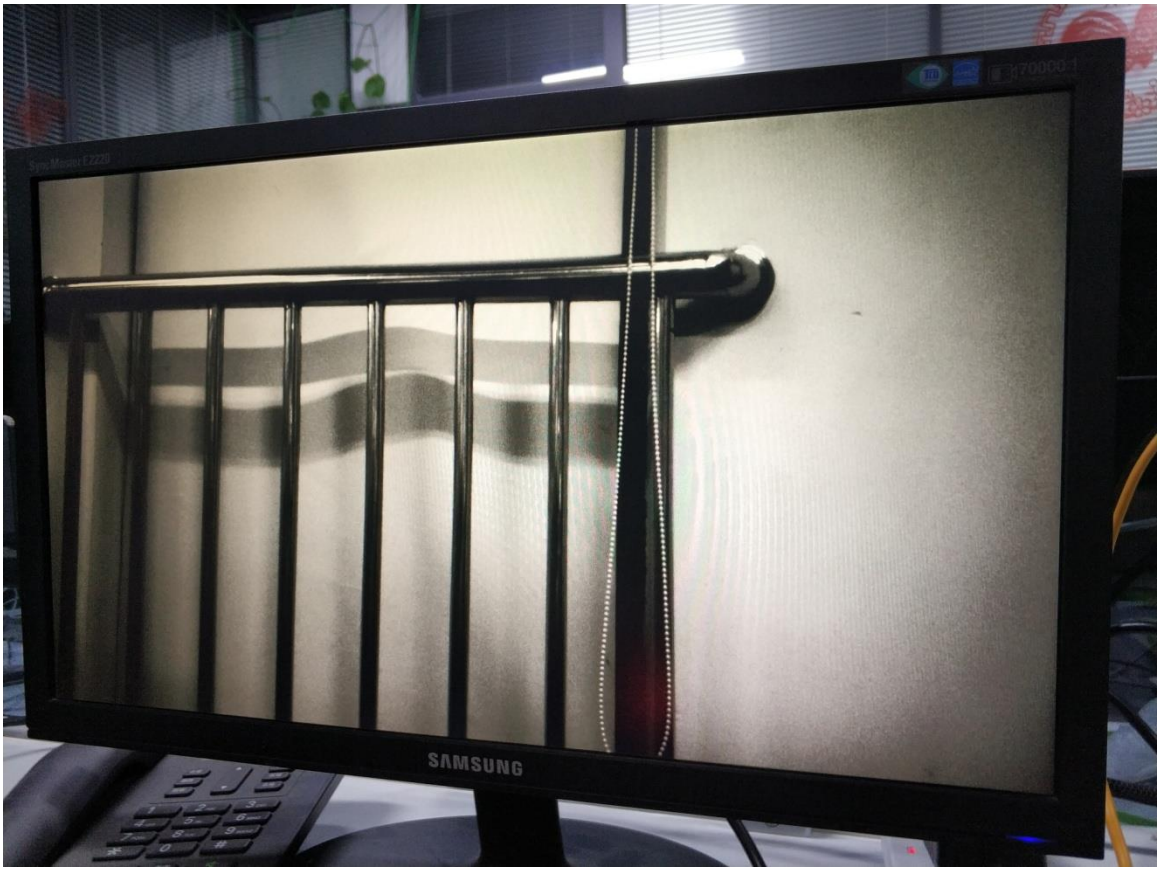
8.4 实验结果

对比度由 0 至 100 由 SDK 动态调整，0 为原图，100 为对比度最高。

对比度为 0 时



对比度为 100 时


SAMSUNG

第九章 自动聚焦

9.1 实验介绍

相机的自动对焦要求相机根据拍摄环境和场景的变化，通过相机内部的微型驱动马达，自动调节相机镜头和 CMOS 之间的距离，保证像平面正好投影到 CMOS 的成像表面上。这时候物体的成像比较清晰，图像细节信息丰富。

图像清晰度评价算法有很多种，在空域中，主要思路是考察图像的领域对比度，即相邻像素间的灰度特征的梯度差；在频域中，主要思路是考察图像的频率分量，对焦清晰的图像高频分量较多，对焦模糊的图像低频分量较多。

本实验选用 Tenengrad 梯度方法，利用 Sobel 算子分别计算水平和垂直方向的梯度，同一场景下梯度值越高，图像越清晰。

我们将焦点由远及近调整，并比较各个焦点位置的 Sobel 结果，值越大则表示该点离最佳聚焦点越近。

因为有的镜头不支持自动调整焦点，对于不支持调焦的镜头，需要手动调整焦点，通过评估值的显示，来确定最佳焦点。

9.2 代码

```
#include "ustream.h"

#define IMG_MAX_HEIGHT    1080
#define IMG_MAX_WIDTH     1920

template<int _T, int ROW, int COL>
void sum_of_stream(hls::Mat<ROW, COL, _T> &_src, unsigned int &average)
{
hls::Scalar<HLS_MAT_CN(_T), HLS_TNAME(_T)> pix;
unsigned int sum=0;

for(int v=0; v<IMG_MAX_HEIGHT; v++)
{
for(int h=0; h<IMG_MAX_WIDTH; h++)
{
#pragma HLS pipeline
_src >> pix;
sum += pix.val[0];
}
}
average = sum;
}

void sobel_focus(ustream_t &src, ustream_t &dst, unsigned int &average)
{
#pragma HLS INTERFACE axis port=src
#pragma HLS INTERFACE axis port=dst
#pragma HLS INTERFACE s_axilite port=average
#pragma HLS INTERFACE ap_ctrl_none port=return

hls::Mat<IMG_MAX_HEIGHT, IMG_MAX_WIDTH, HLS_8UC3> srcImg, grayImg, sobelImg, split0,
```

```
split1;

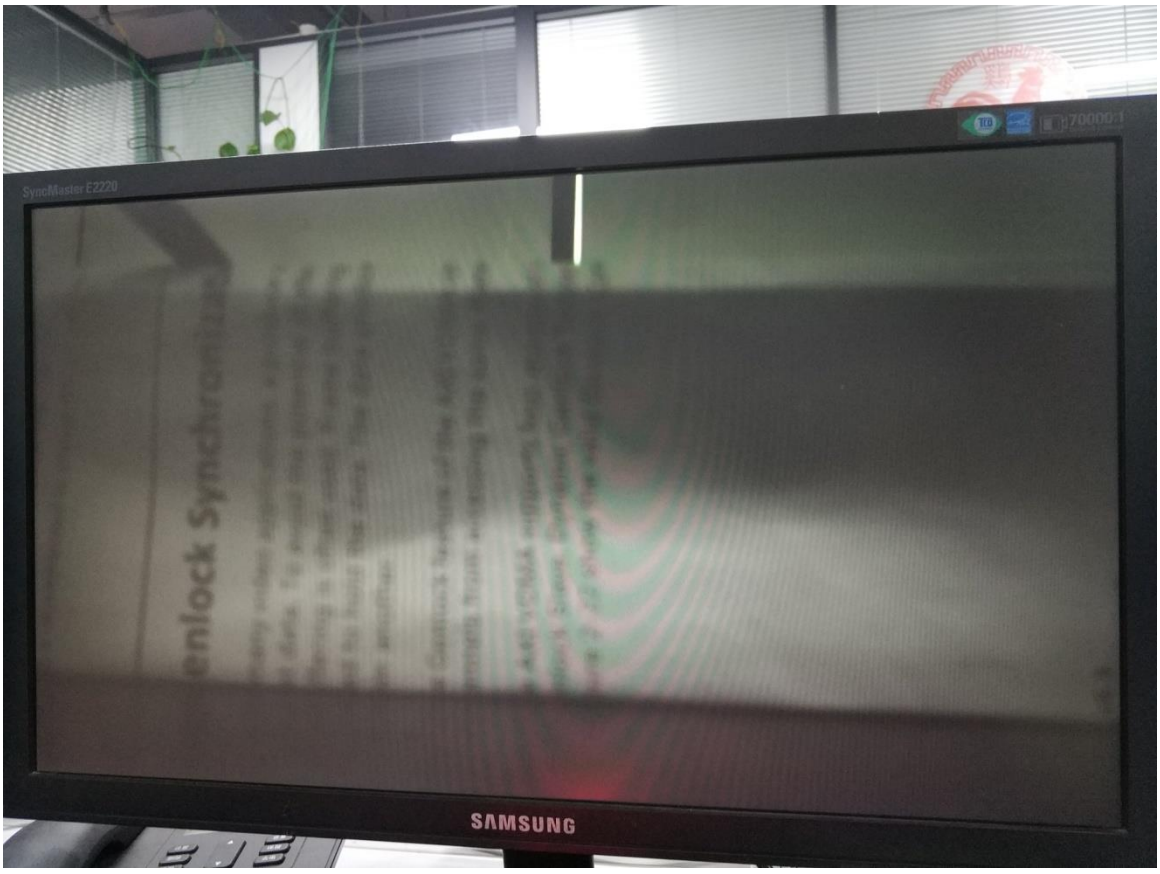
#pragma HLS dataflow
hls::AXIvideo2Mat(src, srcImg);
hls::Duplicate(srcImg, split0, split1);
hls::CvtColor<HLS_RGB2GRAY>(split0, grayImg);
hls::Sobel<1,0,3>(grayImg, sobelImg);
sum_of_stream(sobelImg, average);
hls::Mat2AXIvideo(split1, dst);
}
```

9.3 工程路径

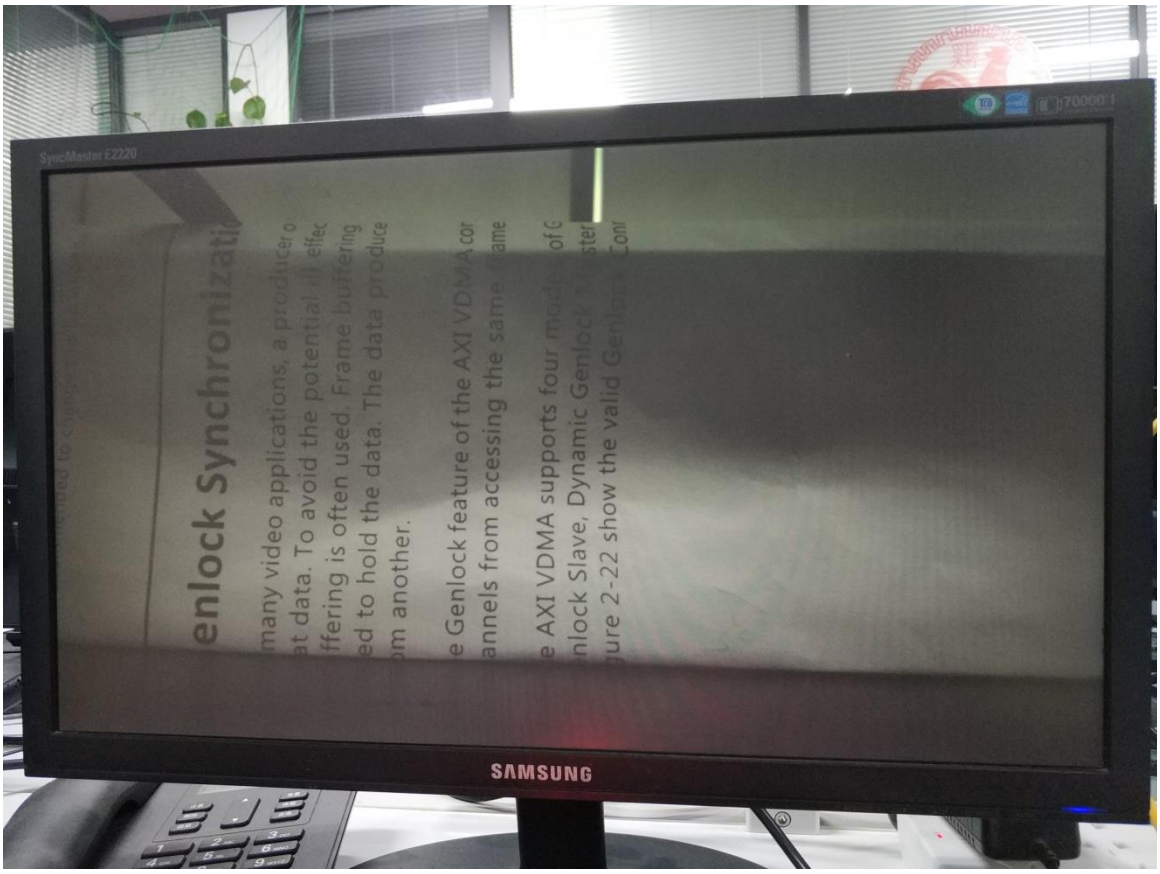
名称	路径
Vivado 工程	vivado/auto_focus
HLS 工程	hls/sobel_focus
HLS 工程	hls/mem2stream
HLS 工程	hls/stream2mem
BOOT.bin 文件	vivado/auto_focus/auto_focus.sdk/app/bootimage/BOOT.bin

9.4 实验结果

默认焦点在最远处，



焦点拉近至最佳点


SyncMaster E2220
enlock Synchroniza
many video applications, a produce
at data. To avoid the potential
ffering is often used. Frame buffering
ed to hold the data. The data produce
m another.
e Genlock feature of the AXI VDMA cor
annels from accessing the same
e AXI VDMA supports four
nlock Slave, Dynamic Genlock
ure 2-22 show the valid
SAMSUNG

第十章 边缘检测

10.1 实验介绍

Sobel 算子，也被称为 Sobel-Feldman 算子，或者 Sobel 滤波，是在图像处理和计算机视觉得到广泛应用的一种图像边缘检测算法。它由斯坦福大学人工智能实验室（SAIL）的 Irwin Sobel 和 Gray Feldman 而得名。

Sobel 算子使用两个（3x3）矩阵来对原图进行卷积运算以计算出两个方向的灰度差分（偏导）的估计值（一个水平方向、一个竖直方向）。我们假定 A 是原始图像（彩色图像需先转换为灰度图像），Gx 和 Gy 分别是在横向及纵向的灰度偏导的近似值（即两个方向上对原图的平面卷积结果）。数学表达如下：

$$\mathbf{G}_x = \begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix} * \mathbf{A} \quad \text{and} \quad \mathbf{G}_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} * \mathbf{A}$$

对应的计算过程如下：

Gx = [f(x+1,y-1)+2*f(x+1,y)+f(x+1,y+1)] - [f(x-1,y-1)+2*f(x-1,y)+f(x-1,y+1)]

Gy = [f(x-1,y-1) + 2f(x,y-1) + f(x+1,y-1)] - [f(x-1, y+1) + 2*f(x,y+1)+f(x+1,y+1)

上式中，f(x,y)为图像 A 中(x,y)处的灰度值。由此便可以计算出每个点的 Gx 和 Gy。

对于图像中的每个点，其梯度的估计值 G 便可以通过两个方向的梯度 Gx 和 Gy 借由下式得出：

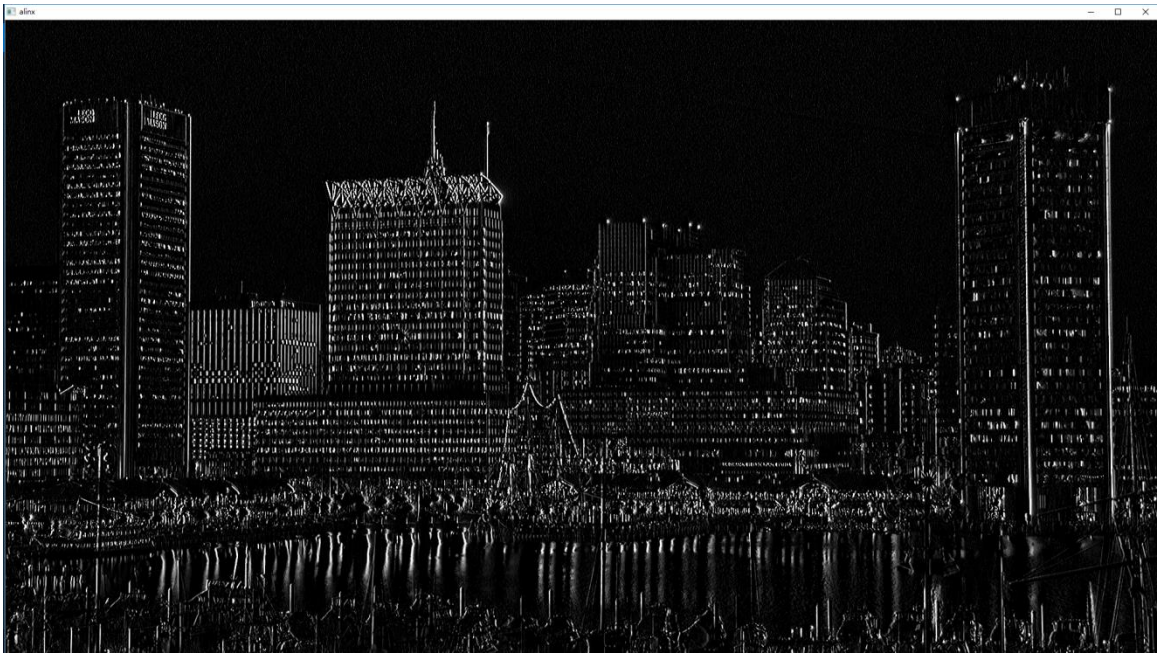
$$\mathbf{G} = \sqrt{\mathbf{G}_x^{\,2} + \mathbf{G}_y^{\,2}}$$

10.2 TestBench 结果

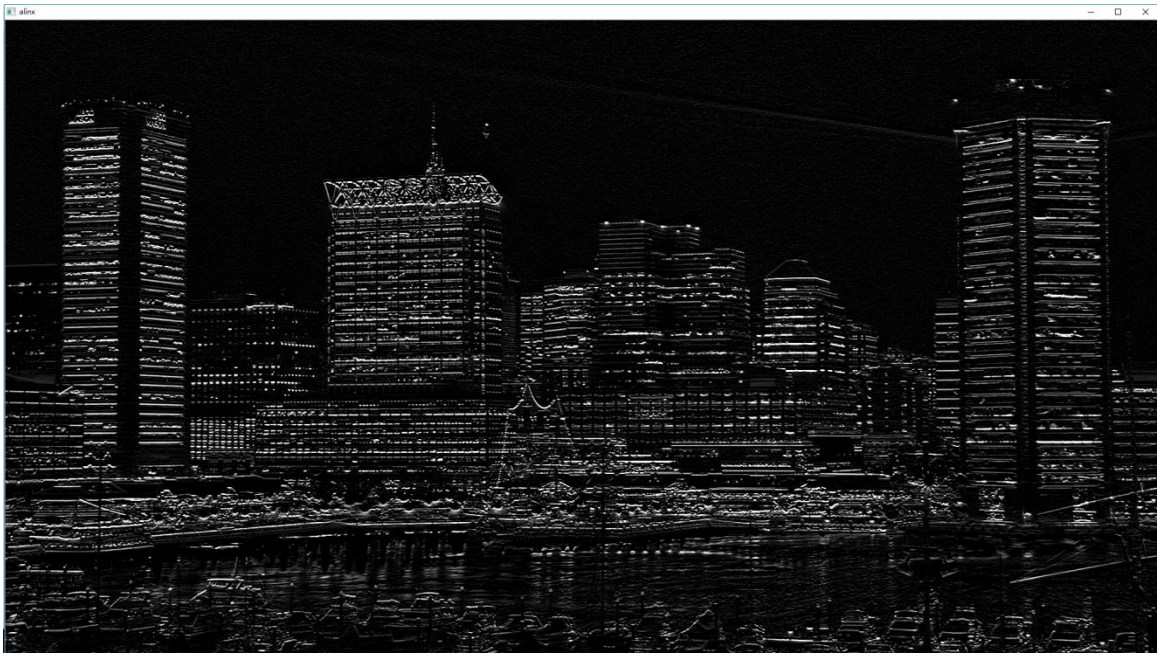
原始图片



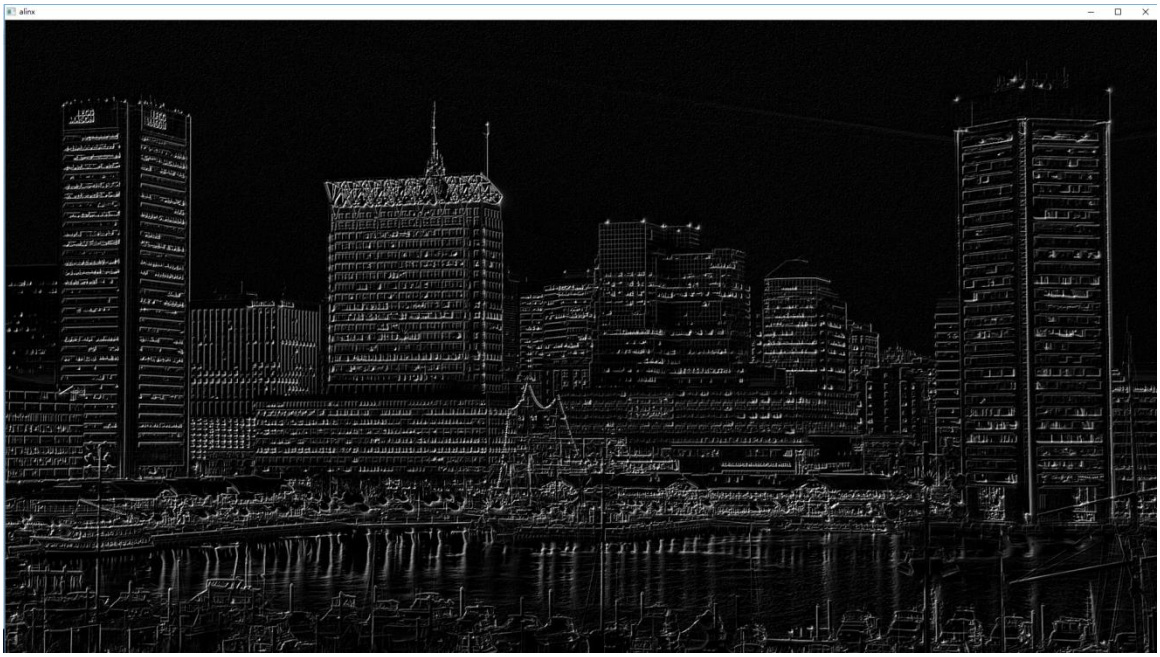
水平方向卷积运算结果



垂直方向卷积运算结果



最终运算结果



10.3 模块主要代码

```
void edge_detector(ustream_t &src, ustream_t &dst, unsigned char threshold)
{
#pragma HLS INTERFACE axis port=src
#pragma HLS INTERFACE axis port=dst
#pragma HLS INTERFACE s_axilite port=threshold
#pragma HLS INTERFACE ap_ctrl_none port=return

	hls::Window<3, 3, ap_int<8> > kernel_x;
	hls::Window<3, 3, ap_int<8> > kernel_y;
	hls::Point_<int> anchor;

	anchor.x = -1;
```

```
        anchor.y = -1;
        kernel_x.val[0][0] =  1;  kernel_x.val[0][1] =  0;  kernel_x.val[0][2] = -1;
        kernel_x.val[1][0] =  2;  kernel_x.val[1][1] =  0;  kernel_x.val[1][2] = -2;
        kernel_x.val[2][0] =  1;  kernel_x.val[2][1] =  0;  kernel_x.val[2][2] = -1;

        kernel_y.val[0][0] =  1;  kernel_y.val[0][1] =  2;  kernel_y.val[0][2] =  1;
        kernel_y.val[1][0] =  0;  kernel_y.val[1][1] =  0;  kernel_y.val[1][2] =  0;
        kernel_y.val[2][0] = -1;  kernel_y.val[2][1] = -2;  kernel_y.val[2][2] = -1;

        hls::Mat<IMG_MAX_HEIGHT, IMG_MAX_WIDTH, HLS_8UC3> srcImg, rgbSobel;
        hls::Mat<IMG_MAX_HEIGHT, IMG_MAX_WIDTH, HLS_8UC1> grayImg, sobelImg_x, sobelImg_y, sobelImg, thresholdImg, split0, split1;

#pragma HLS dataflow
        hls::AXIvideo2Mat(src, srcImg);
        hls::CvtColor<HLS_RGB2GRAY>(srcImg, grayImg);
        hls::Duplicate(grayImg, split0, split1);
        hls::Filter2D(split0, sobelImg_x, kernel_x, anchor);
        hls::Filter2D(split1, sobelImg_y, kernel_y, anchor);
        hls::AddWeighted(sobelImg_x,0.5f,sobelImg_y,0.5f,0.0f,sobelImg);
        thresholding(thresholdImg, sobelImg, threshold);
        hls::CvtColor<HLS_GRAY2RGB>(thresholdImg, rgbSobel);
        hls::Mat2AXIvideo(rgbSobel, dst);
}
```

这里 threshold 为将结果二值化的配置，为 0 时不进行二值化，其它值时为二值化的阀值。我们首选将 rgb 图像转换为灰度图 grayImg，将 grayImg 分成两份 split0、split1 分别进行水平和垂直方向的卷积运算，得到 sobelImg_x 和 sobelImg_y，将它们叠加为 sobelImg。然后将其二值化，转为 RGB，输出。

10.4 工程路径

名称	路径
Vivado 工程	vivado/ edge_detector
HLS 工程	hls/ edge_detector
生成结果	vivado/ edge_detector/edge_detector.sdk/app/bootimage/BOOT.bin

10.5 实验结果

Threshold 设置为 0 时



Threshold 设置为 20 时



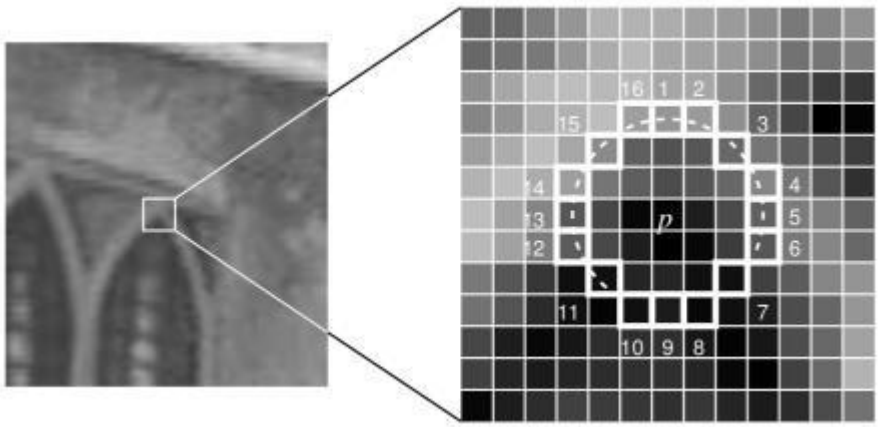
第十一章 角点检测

角点检测(Corner Detection)是计算机视觉系统中用来获得图像特征的一种方法，广泛应用于运动检测、图像匹配、视频跟踪、三维建模和目标识别等领域中。也称为特征点检测。

FAST（Features fromaccelerated segment test）是一种角点检测方法，最初是由 Edward Rosten 和 Tom Drummond 提出，该算法最突出的优点是它的计算效率。正如它的缩写名字，它很快而且事实上它比其他著名的特征点提取方法（如 SIFT，SUSAN，Harris）都要快。如果应用于机器学习方法的话，该算法能够取得更佳的效果。正因为它的快速特点，FAST 角点检测方法非常适用于实时视频处理的领域。

FAST 算法步骤

1) 从图片中选取一个像素 P，下面我们将判断它是否是一个特征点。我们首先把它的亮度值设为 Ip。
2) 设定一个合适的阈值 t。
3) 考虑以该像素点为中心的一个半径等于 3 像素的离散化的 Bresenham 圆，这个圆的边界上有 16 个像素（如图 1 所示）。



4) 现在，如果在这个大小为 16 个像素的圆上有 n 个连续的像素点，它们的像素值要么都比 Ip+t 大，要么都比 Ip−t 小，那么它就是一个角点。（如图 1 中的白色虚线所示）。n 的值可以设置为 12 或者 9，实验证明选择 9 可能会有更好的效果。

上面的算法中，对于图像中的每一个点，我们都要去遍历其邻域圆上的 16 个点的像素，效率较低。我们下面提出了一种高效的测试（high-speed test）来快速排除一大部分非角点的像素。该方法仅仅检查在位置 1，9，5 和 13 四个位置的像素，首先检测位置 1 和位置 9，如果它们都比阈值暗或比阈值亮，再检测位置 5 和位置 13。如果 PP 是一个角点，那么上述四个像素点中至少有 3 个应该必须都大于 Ip+t 或者小于 Ip−t，因为若是一个角点，超过四分之三圆的部分应该满足判断条件。如果不满足，那么 p 不可能是一个角点。对于所有点做上面这一部分初步的检测后，符合条件的将成为候选的角点，我们再对候选的角点，做完整的测试，即检测圆上的所有点。

上面的算法效率实际上是很高的，但是有一些缺点：

1) 当我们设置 n<12 时就不能使用快速算法来过滤非角点的点；

2) 检测出来的角点不是最优的，这是因为它的效率取决于问题的排序与角点的分布;
3) 对于角点分析的结果被丢弃了;
4) 多个特征点容易挤在一起。

可以通过机器学习和非极大值抑制的方法来增强鲁棒性。由于 opencv 中相关的函数没有使用机器学习，因此我们这里只介绍非极大值抑制的方法。由于分割测试并没有计算角点响应函数，因此常规的非极大值抑制方法并不适用于 FAST 算法。下面是 FAST 的非极大值抑制方法:

1) 计算得分函数，它的值 V 是特征点与其圆周上 16 个像素点的绝对差值中所有连续 10 个像素中的最小值的最大值，而且该值还要大于阈值 t;
2) 在 3×3 的特征点邻域内（而不是图像邻域），比较 V;
3) 剔除掉非极大值的特征点。

在 opencv 中，实现 FAST 算法的核心函数有两个，它们的原型为:

void FAST(InputArray image, vector<KeyPoint>& keypoints, int threshold, bool nonmaxSuppression=true)

void FASTX(InputArray image, vector<KeyPoint>& keypoints, int threshold, bool nonmaxSuppression, int type)

image 为输入图像，要求是灰度图像，keypoints 为检测到的特征点向量，threshold 为阈值 t，nonmaxSuppression 为是否进行非极大值抑制，true 表示进行非极大值抑制，type 为选取圆周像素点的个数，是 8（FastFeatureDetector::TYPE_5_8）、12（FastFeatureDetector::TYPE_7_12）还是 16（FastFeatureDetector::TYPE_9_16）。该参数是 FAST 函数和 FASTX 函数的区别，事实上，FAST 函数是调用 FASTX 函数，而传入的 type 值为 FastFeatureDetector::TYPE_9_16。

本节实验，我们创建一个 detectCorner 的 HLS 模块，从 DDR 读取视频图像，检测角点后，将结果写回 DDR。

在 SDK 中，我们将写回 DDR 的角点检测结果，叠加在显示的图像上。

11.1 模块主要代码

```
void detectCorner(unsigned int *pImgRGB888, unsigned int *pCorner, int rows, int cols, int threshold)
{
#pragma HLS INTERFACE m_axi port=pImgRGB888    offset=slave bundle=img
#pragma HLS INTERFACE m_axi port=pCorner offset=slave bundle=corner
#pragma HLS INTERFACE s_axilite port=rows
#pragma HLS INTERFACE s_axilite port=cols
#pragma HLS INTERFACE s_axilite port=threshold
#pragma HLS INTERFACE s_axilite port=return

hls::Mat<IMG_MAX_ROWS, IMG_MAX_COLS, HLS_8UC3> matSrc(rows, cols);
hls::Mat<IMG_MAX_ROWS, IMG_MAX_COLS, HLS_8UC1> matGray(rows, cols);
hls::Mat<IMG_MAX_ROWS, IMG_MAX_COLS, HLS_8UC1> mask(rows, cols);
hls::Mat<IMG_MAX_ROWS, IMG_MAX_COLS, HLS_8UC1> dmask(rows, cols);
hls::Scalar<3,unsigned char> color(255,0,0);

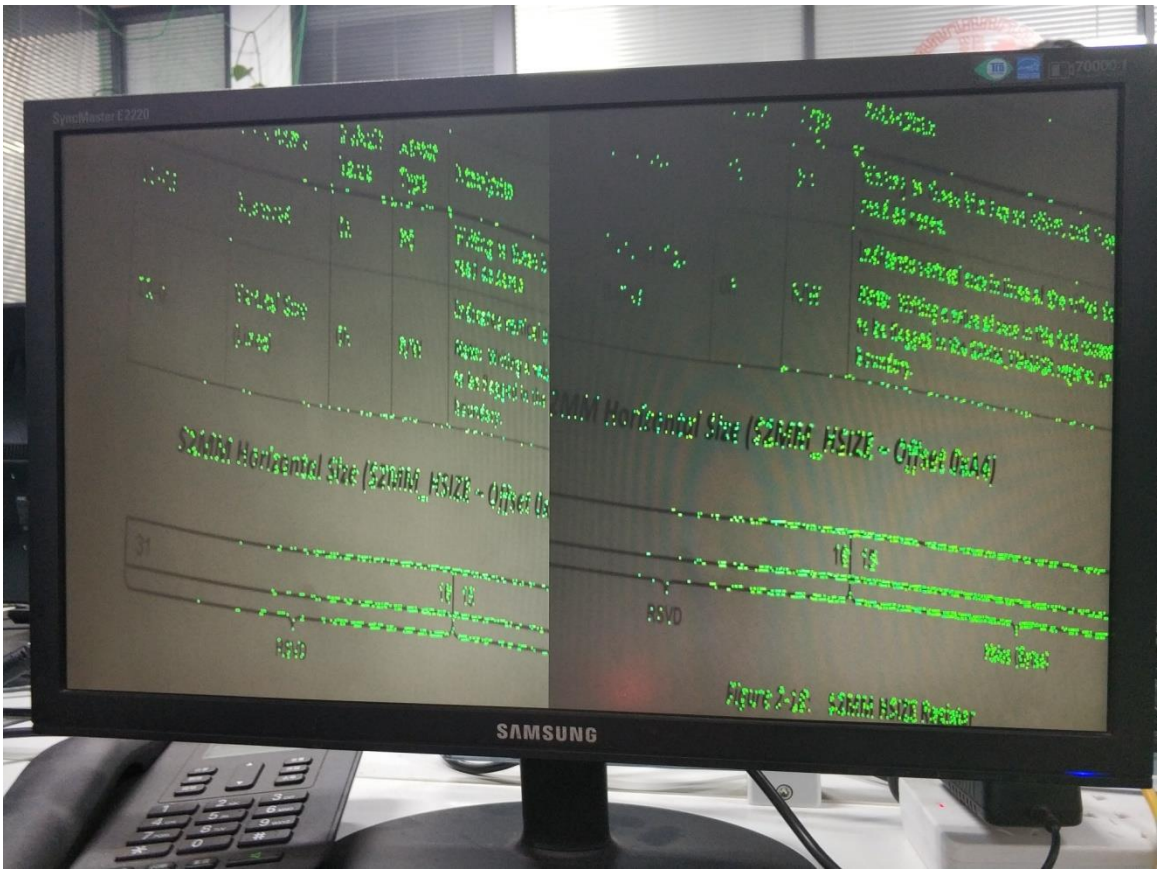
#pragma HLS dataflow
createImg(pImgRGB888, matSrc);
```

```
hls::CvtColor<HLS_RGB2GRAY>(matSrc, matGray);
hls::FASTX(matGray, mask, threshold, true);
hls::Dilate(mask, dmask);
writeBack(pCorner, dmask);
}
```

11.2 工程路径

名称	路径
Vivado 工程	vivado/dual_corner
HLS 工程	hls/detectCorner
HLS 工程	hls/overlayosd
HLS 工程	hls/mem2stream
HLS 工程	hls/stream2mem
BOOT.bin 文件	vivado/dual_corner/dual_corner.sdk/app/bootimage/BOOT.bin

11.3 实验结果



这里角点多少与聚焦位置及阀值有关。需要根据场景调整这两个参数。

第十二章 快速傅里叶变换 FFT

12.1 实验介绍

FFT 是离散傅立叶变换的快速算法，可以将一个信号变换到频域。N 个采样点，经过 FFT 之后，就可以得到 N 个复数结果。为了方便进行 FFT 运算，通常 N 取 2 的整数次方。这里我们设定最大为 1024，即 2 的 10 次方。

假设采样频率为 Fs，信号频率 F，采样点数为 N。那么 FFT 之后结果就是一个为 N 点的复数。每一个点就对应着一个频率点，即 Fn=(n-1)*Fs/N。这个点的模值，就是该频率值下的幅度特性。

假设 FFT 之后某点 n 用复数 a+bi 表示，那么这个复数的模就是 An= $\sqrt{a*a+b*b}$，相位就是 Pn=atan2(b,a)。根据以上的结果，就可以计算出 n 点（n≠1，且 n<=N/2）对应的信号的表达式为：An/(N/2)*cos(2*pi*Fn*t+Pn)，即 2*An/N*cos(2*pi*Fn*t+Pn)。对于 n=1 点的信号，是直流分量，幅度即为 A1/N。由于 FFT 结果的对称性，通常我们只使用前半部分的结果，即小于采样频率一半的结果。

模块首先使用指定的内存数据 pSrc 初始化 xn 数组，接下来调用 fft 完成变换，结果存放在数组 xk 中，最后，我们计算模值并拷贝至 pDst。这里模值因为 scale 关系，需要乘 2048。

12.2 模块主要代码

```
void getfft(int nfft, unsigned short *pSrc, unsigned short *pDst)
{
#pragma HLS INTERFACE s_axilite port=nfft
#pragma HLS INTERFACE m_axi port=pSrc offset=slave
#pragma HLS INTERFACE m_axi port=pDst offset=slave
#pragma HLS INTERFACE s_axilite port=return
#pragma HLS dataflow

config_t fft_config;
status_t fft_status;
cmpxDataIn xn[FFT_LENGTH];
cmpxDataOut xk[FFT_LENGTH];

fft_config.setDir(0);
fft_config.setSch(0x2AB);
fft_config.setNfft(nfft);
access_src(pSrc, xn, 1<<nfft);
hls::fft<config1>(xn, xk, &fft_status, &fft_config);
access_dst(pDst, xk, 1<<nfft);
}
```

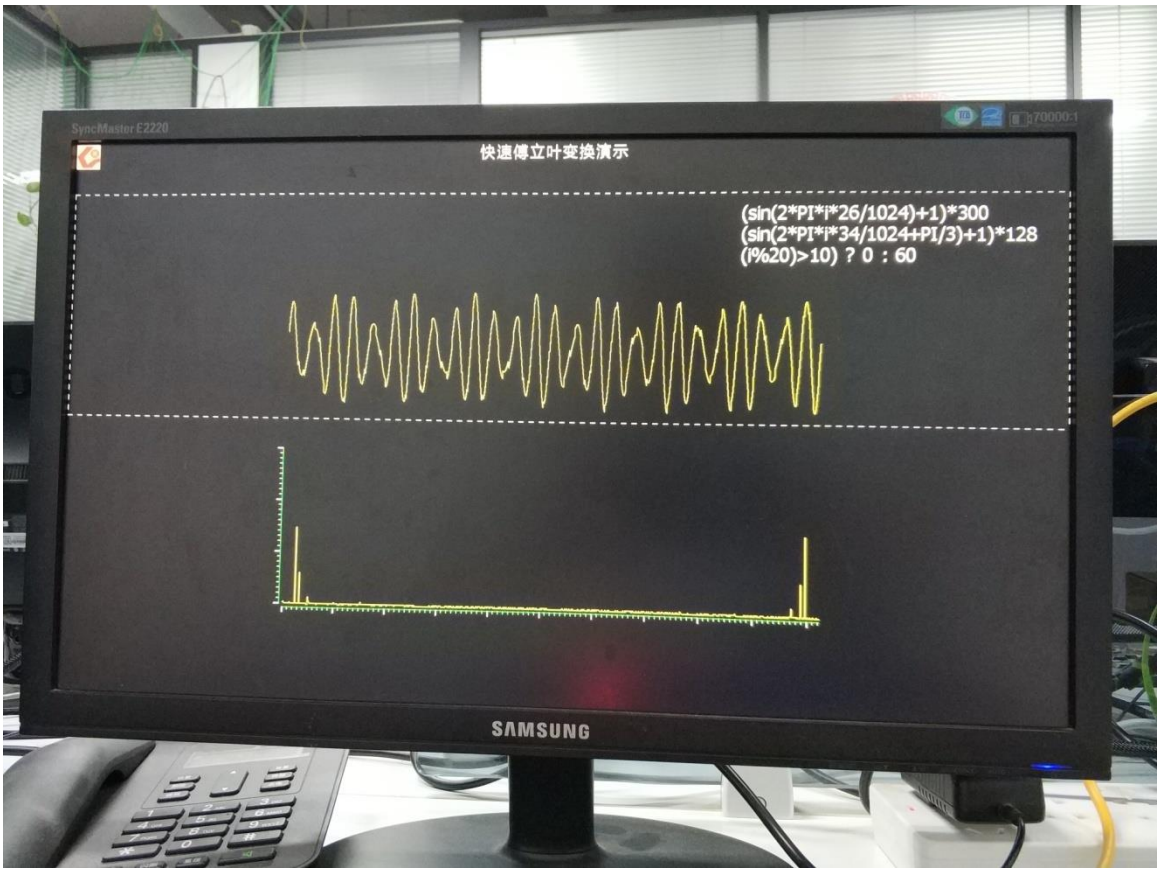
这里，setSch(0x2AB) 是设置 scale，即蝶形算法的每一级右移的位数 [2 2 2 2 3]。这样确保最后的结果也是 16 位的。这里 scale 对应的十进制值是 2^(2+2+2+2+3)=2048。

setNfft 用来设置采样数据长度，这里定义 10，即 2 的 10 次方，表示 1024 个采样点，setDir 用来设置正逆变换。

12.3 工程路径

名称	路径
Vivado 工程	vivado/fft_demo
HLS 工程	hls/getfft
HLS 工程	hls/overlayosd
BOOT.bin 文件	vivado/fft_demo/fft_demo.sdk/app/bootimage/BOOT.bin

12.4 实验结果



虚框中为生成的三个波形叠加波形，虚框下面为 FFT 后结果，这里直流分量没有显示。